

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)

Институт информационных технологий

**Кафедра управления и информатики
в технических системах**

Отчет по выполнению лабораторной работы №5
«Защита Проекта»
по дисциплине «Информационные системы и технологии»

Направление подготовки: 09.03.03 «Прикладная информатика»
Профиль «Киберфизические системы»

Преподаватель:

Бычков Сергей Юрьевич

Оценка

Подпись

22.05.2025_
Дата

Студенты:

Вежновец Дмитрий, Золотухин Иван,

Кужба Полина, Мокроусова Лариса

Подпись

22.05.2025_
Дата

Группа: ИДБ-22-12

Москва, 2025

Цель работы

Сконструировать информационную систему «Виртуальный деканат» с использованием трёхзвенной архитектуры (*Database ↔ Backend ↔ Frontend*), обеспечив её соответствие диаграммам базы данных и функциональной модели, представленной в нотации **IDEF0**. В процессе разработки освоить работу с технологическим стеком **Java + Spring Boot + PostgreSQL** для серверной части и **TypeScript + React + MUI** для клиентского интерфейса, а также научиться документировать и тестировать REST-API с помощью **Postman**.

Используемое программное обеспечение

Таблица 1. Слои и технологии ПО

Слой	Технологии / версии
База данных	PostgreSQL 15, psql CLI
Backend (API)	Java 17, Spring Boot 3, Spring Security, Spring Data JPA, Maven, Lombok
Инструменты теста	Postman v10 (коллекции, окружения)
Frontend	React 18, TypeScript 5, Vite, Material-UI (v5), Redux Toolkit, Framer Motion
Прочее	Docker (+ docker-compose для БД), Git, IntelliJ IDEA / WebStorm

1. Проектирование базы данных

1.1 ER-модель (фрагмент)

INSTITUTE (id PK, name, email, phone)

└─< 1:N >─ KAFEDRA (id PK, institute_id FK, name, room, email, phone, credentials_non_expired)

└─< 1:N >─ EMPLOYEE (id PK, kafedra_id FK, job_title, credentials_non_expired, person_id FK)

STUDENT (id PK, year_started, financial_form, person_id FK)

PERSON (id PK, surname, name, patronymic, phone)

Каждая сущность хранится в отдельной таблице; связи реализованы внешними ключами. Индексы созданы на часто фильтруемых полях (year_started, financial_form, credentials_non_expired).

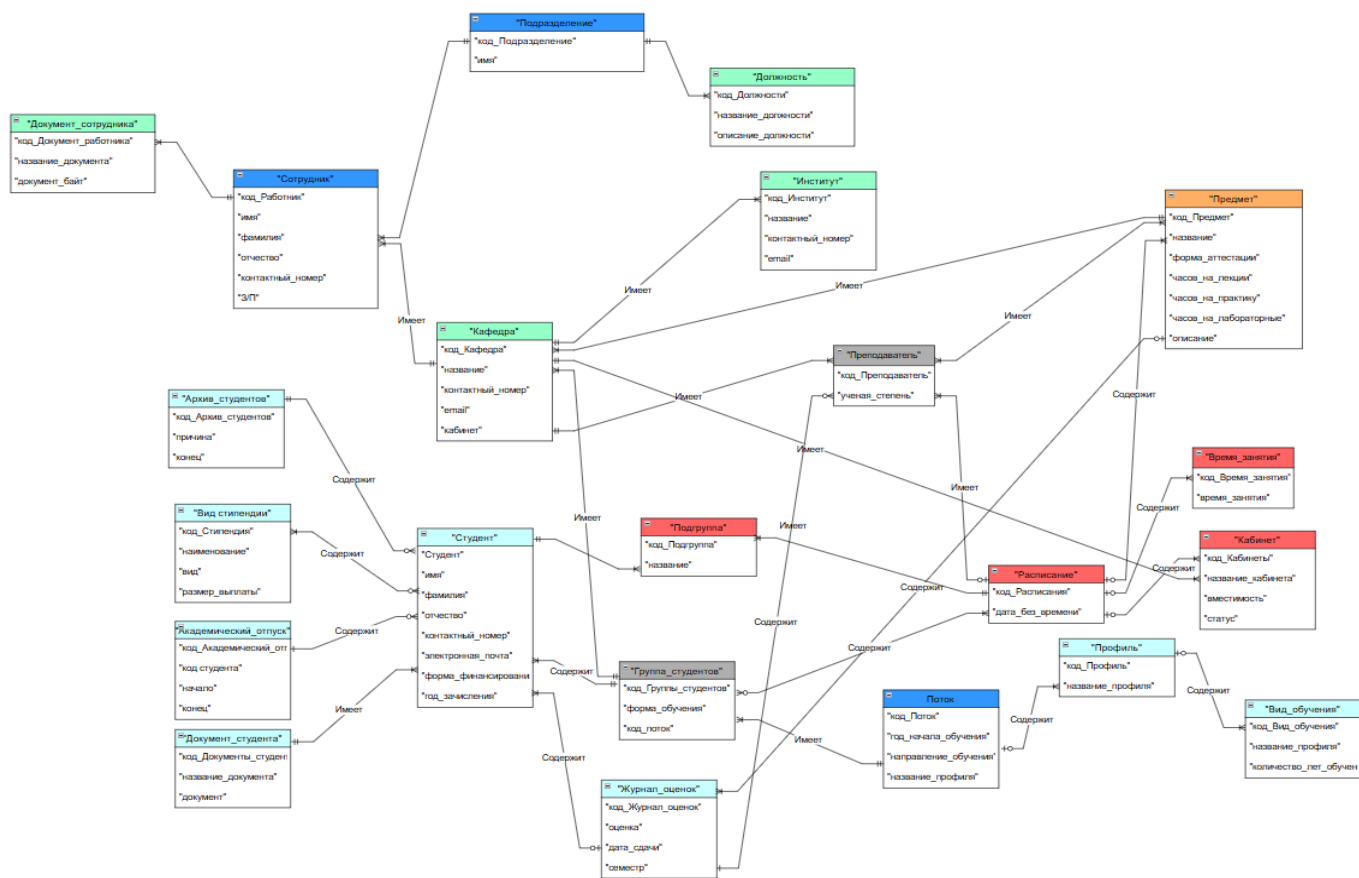


Рис.1 ERA-модель

1.2 IDEF0

Работа со студентами:

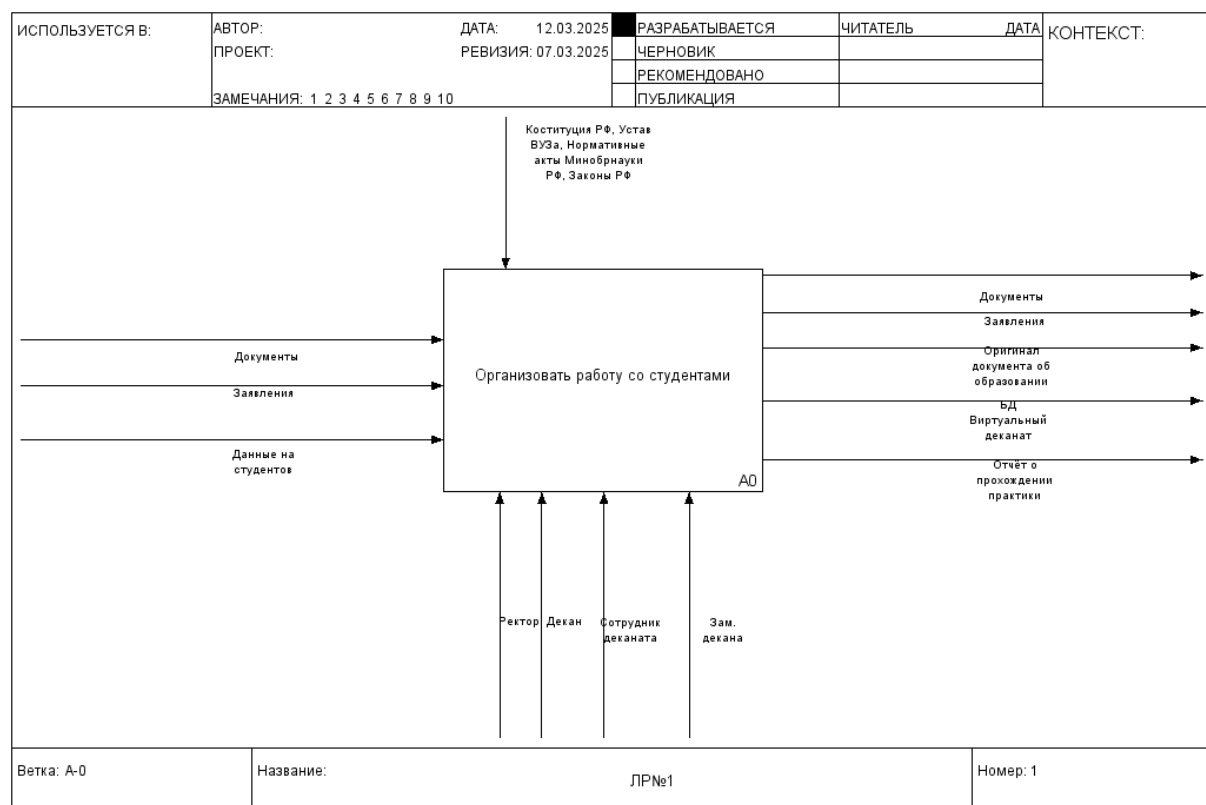


Рис.2 IDEF0 Контекстная диаграмма Организовать работу со студентами

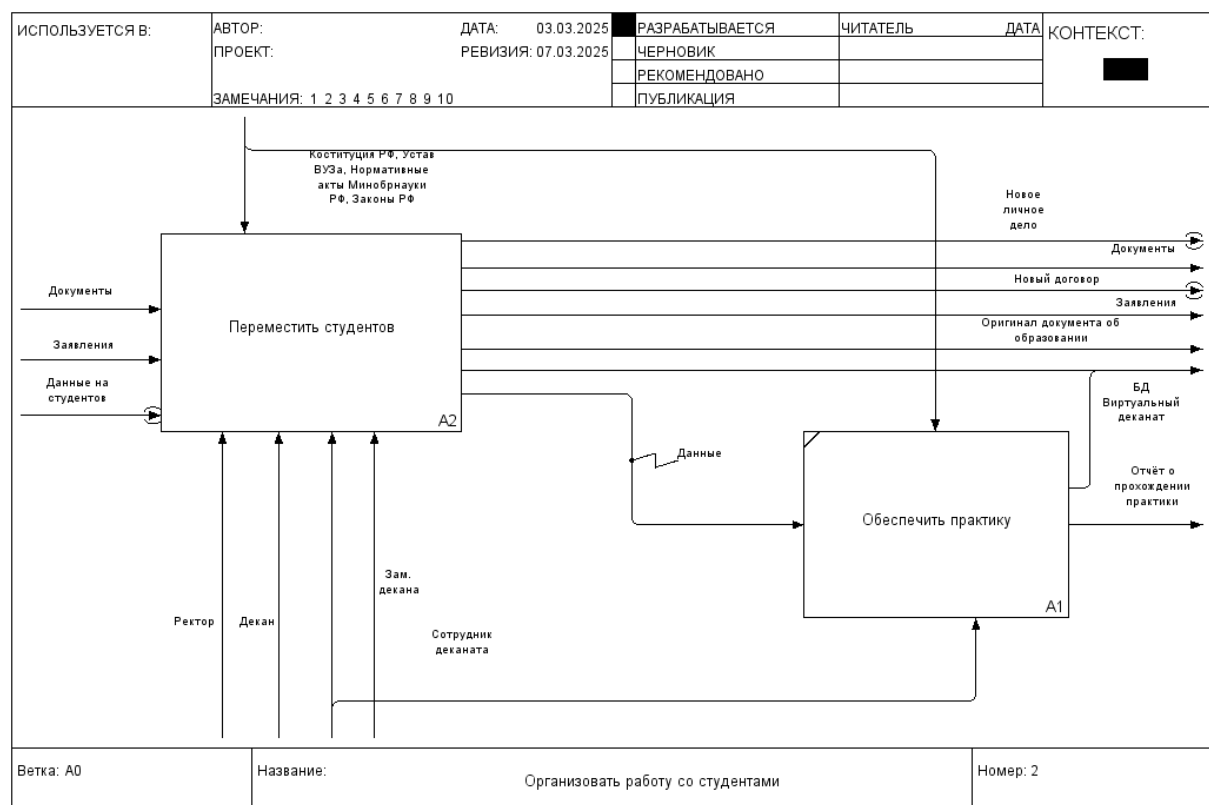


Рис.3 IDEF0 диаграмма декомпозиции первого уровня Организовать работу со студентами

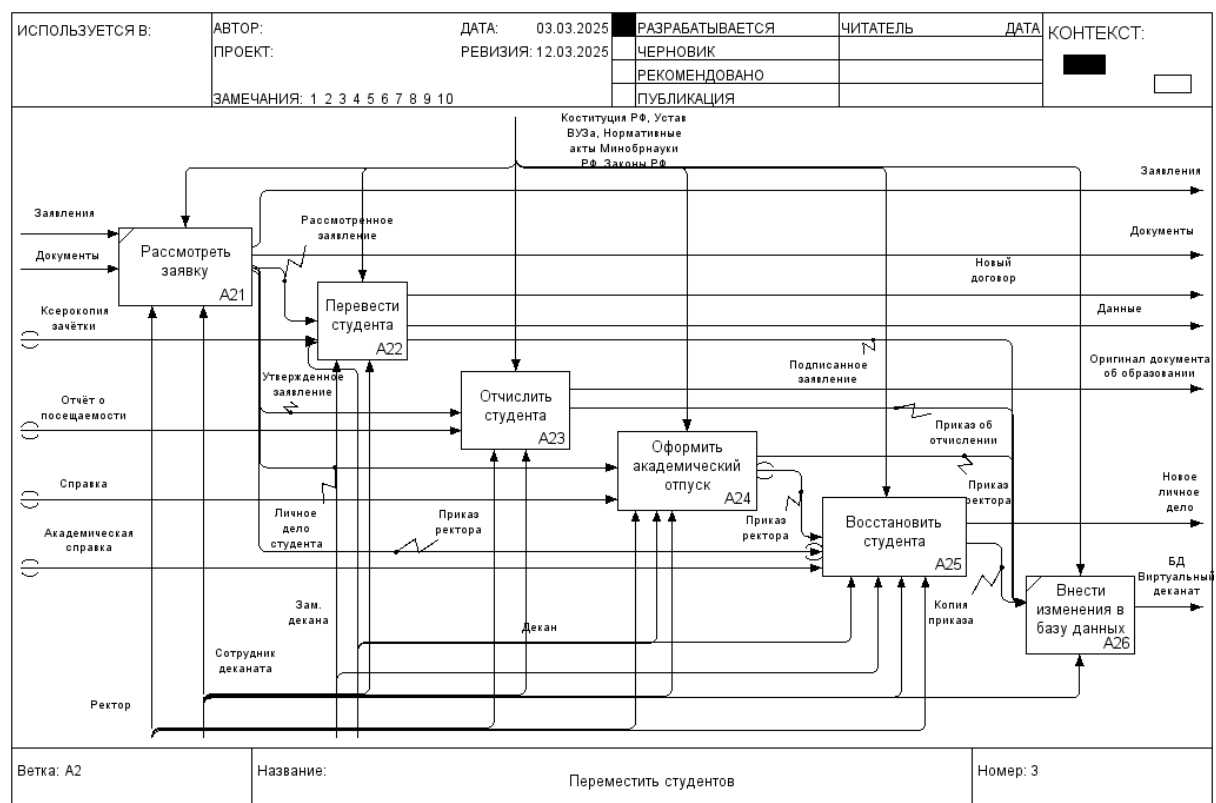


Рис.4 IDEF0 диаграмма декомпозиции второго уровня Переместить студентов

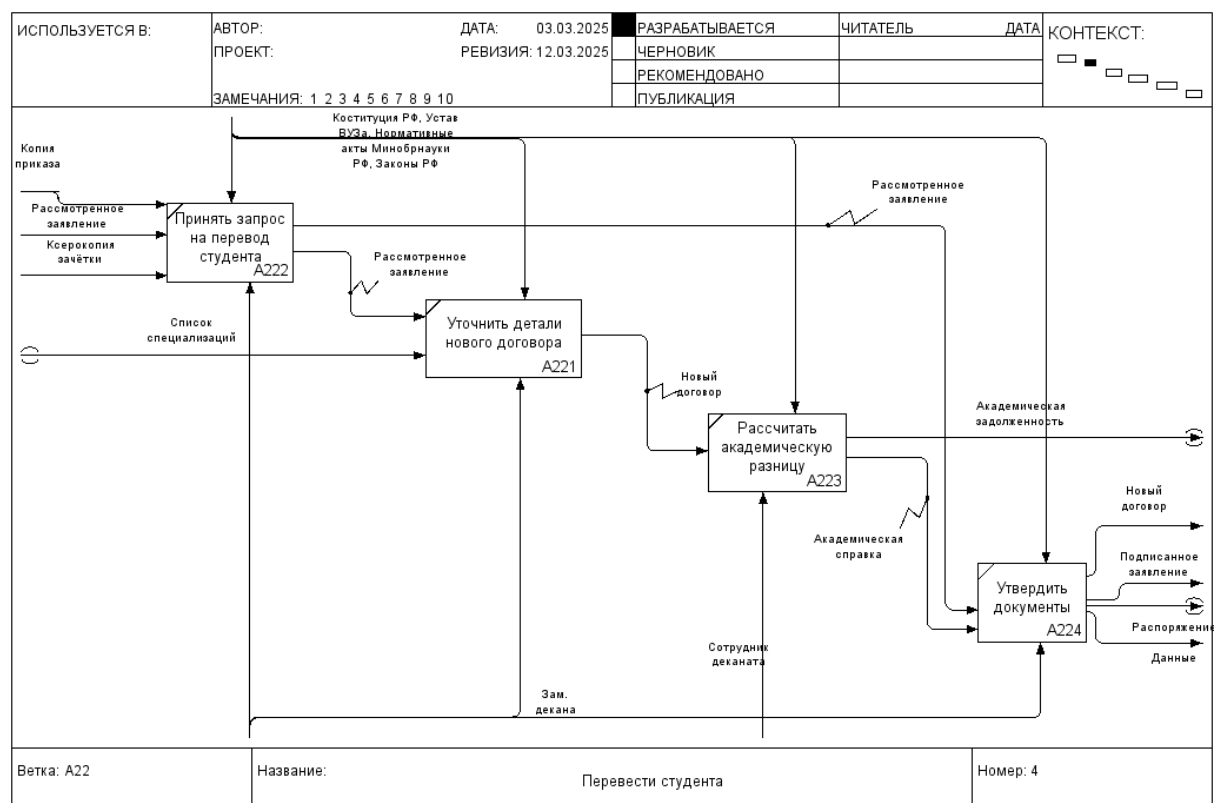


Рис.5 IDEF0 диаграмма декомпозиции третьего уровня Перевести студента

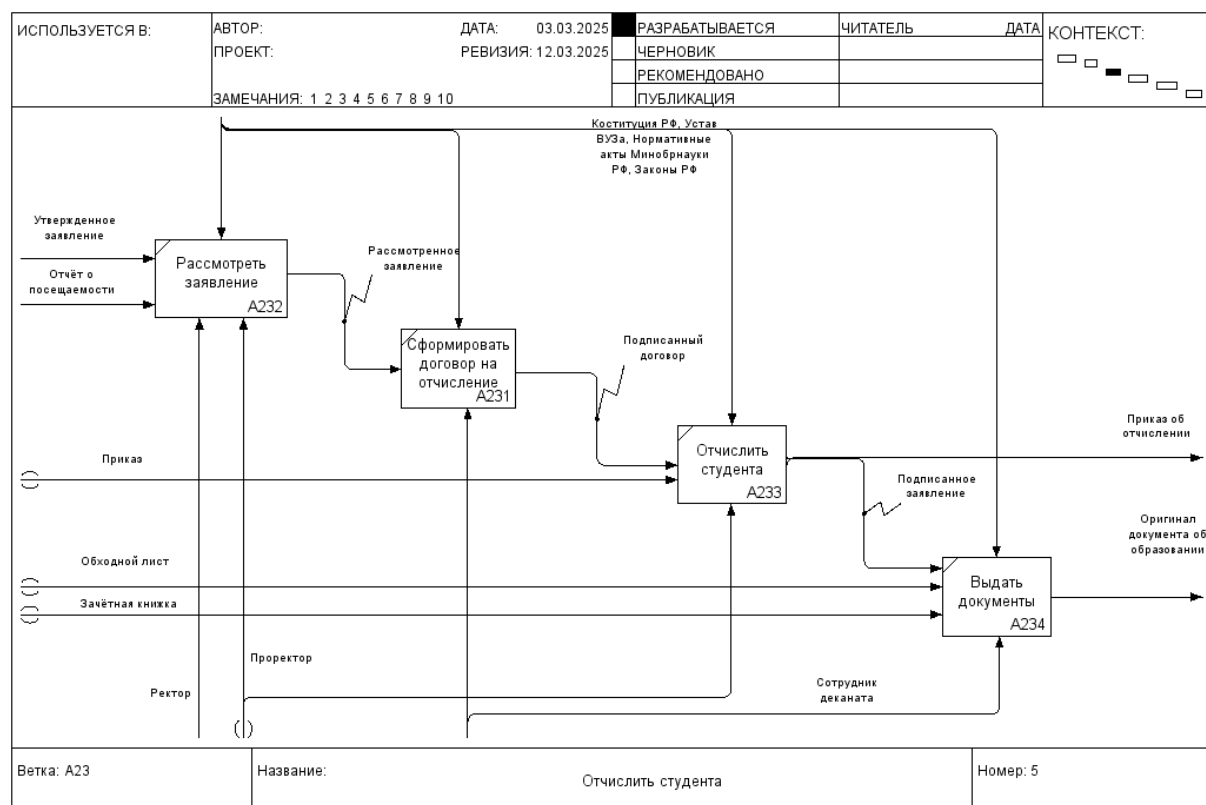


Рис.6 IDEF0 диаграмма декомпозиции третьего уровня Отчислить студента

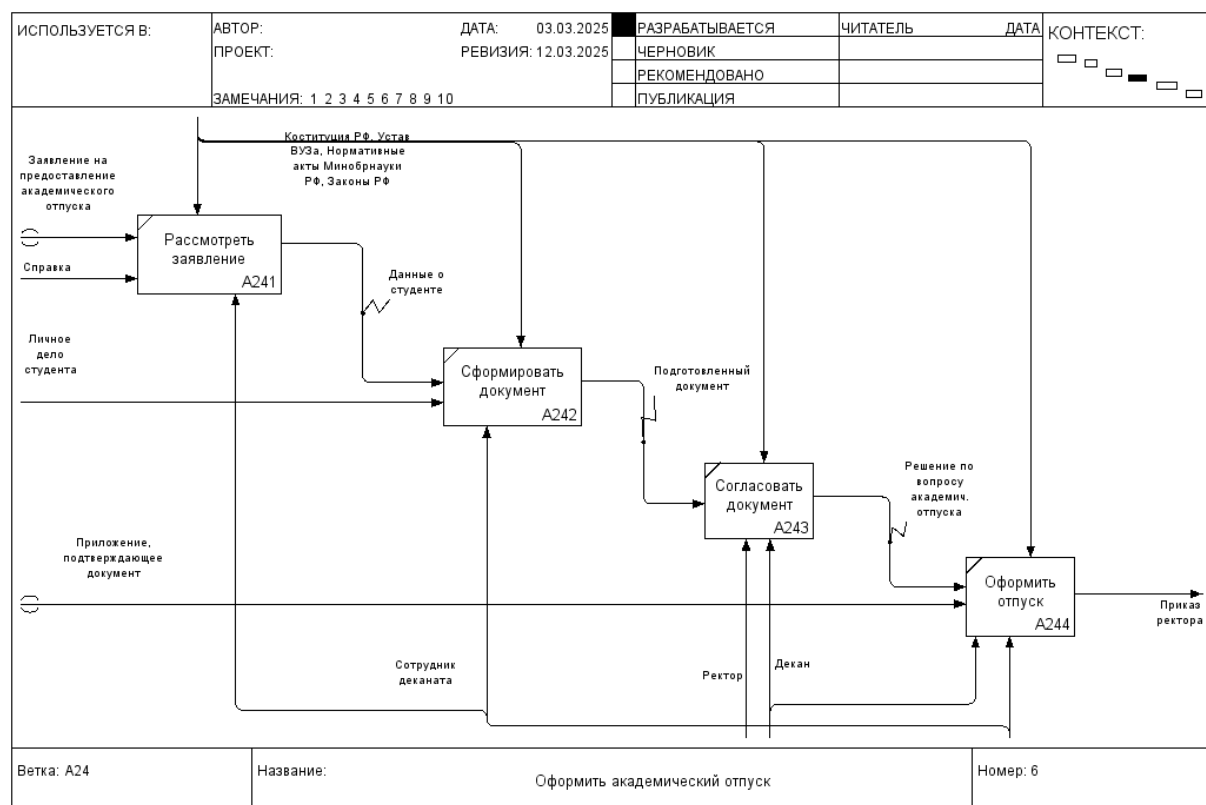


Рис.7 IDEF0 диаграмма декомпозиции третьего уровня Оформить академический отпуск

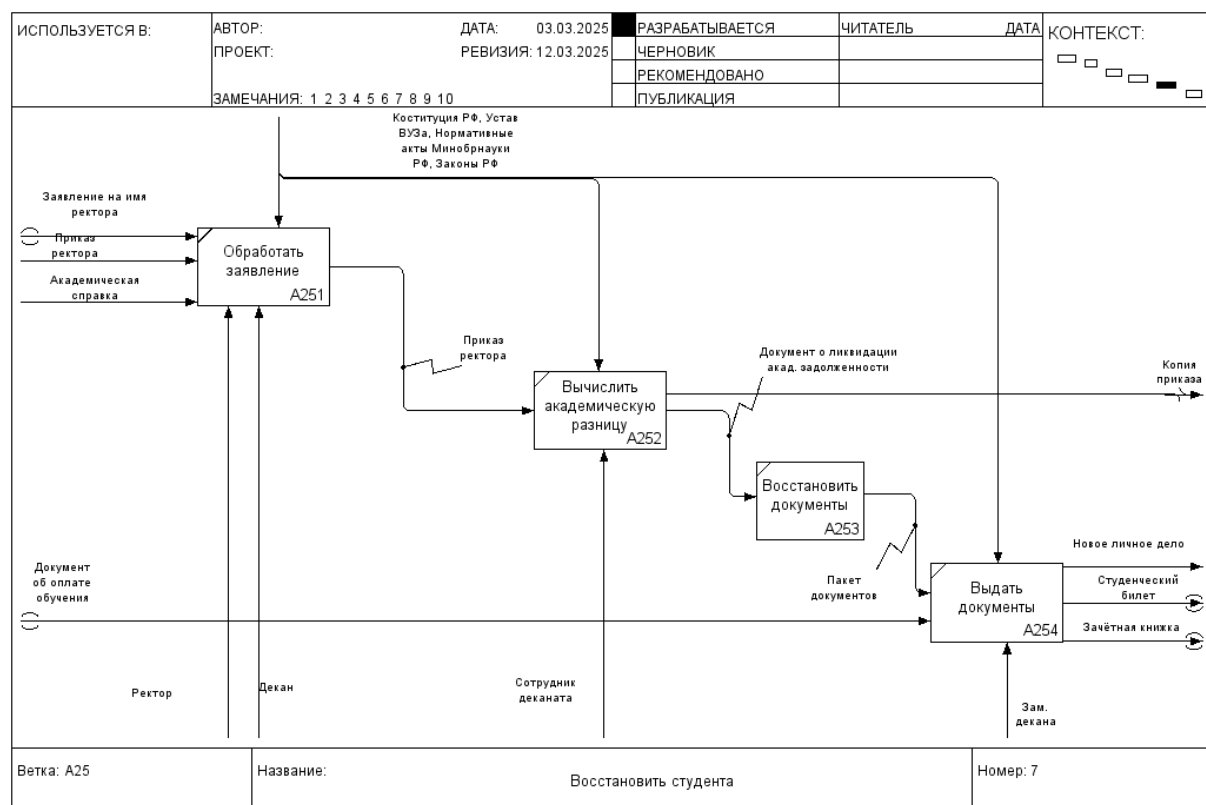


Рис.8 IDEF0 диаграмма декомпозиции третьего уровня Восстановить студента

Кафедры:

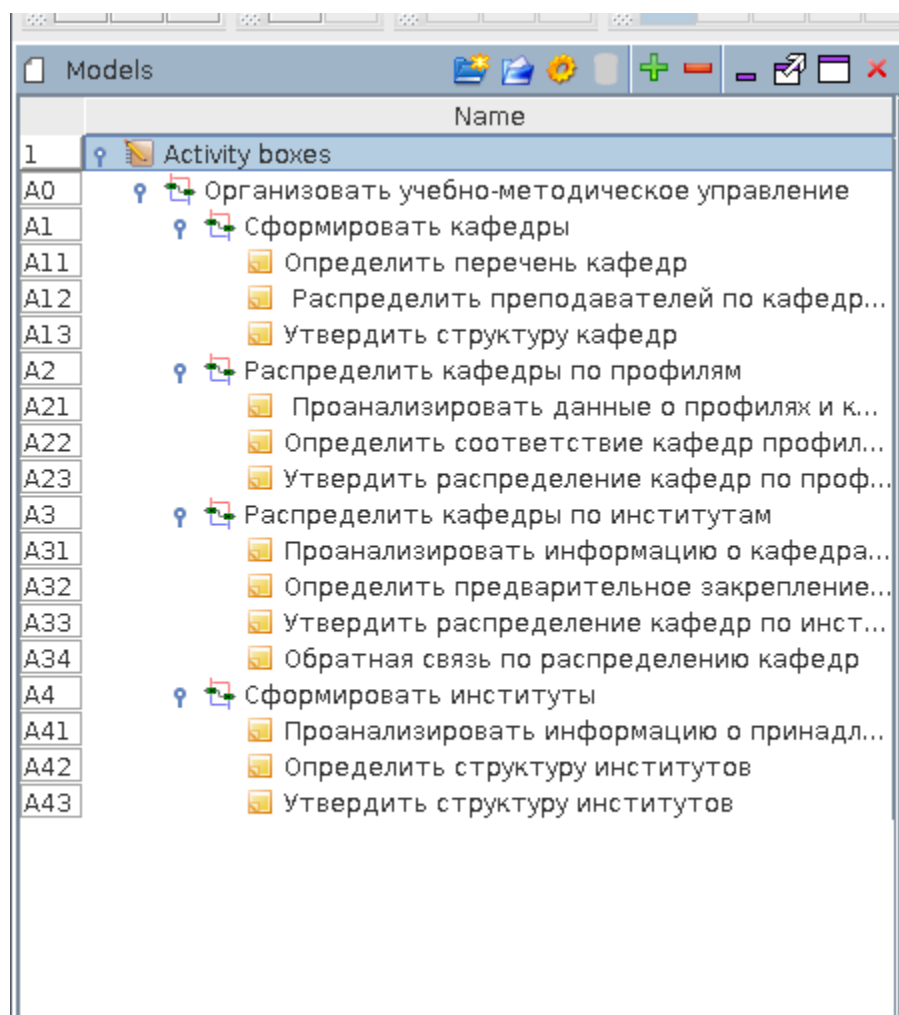


Рис. 9 Иерархия диаграмм IDEF0

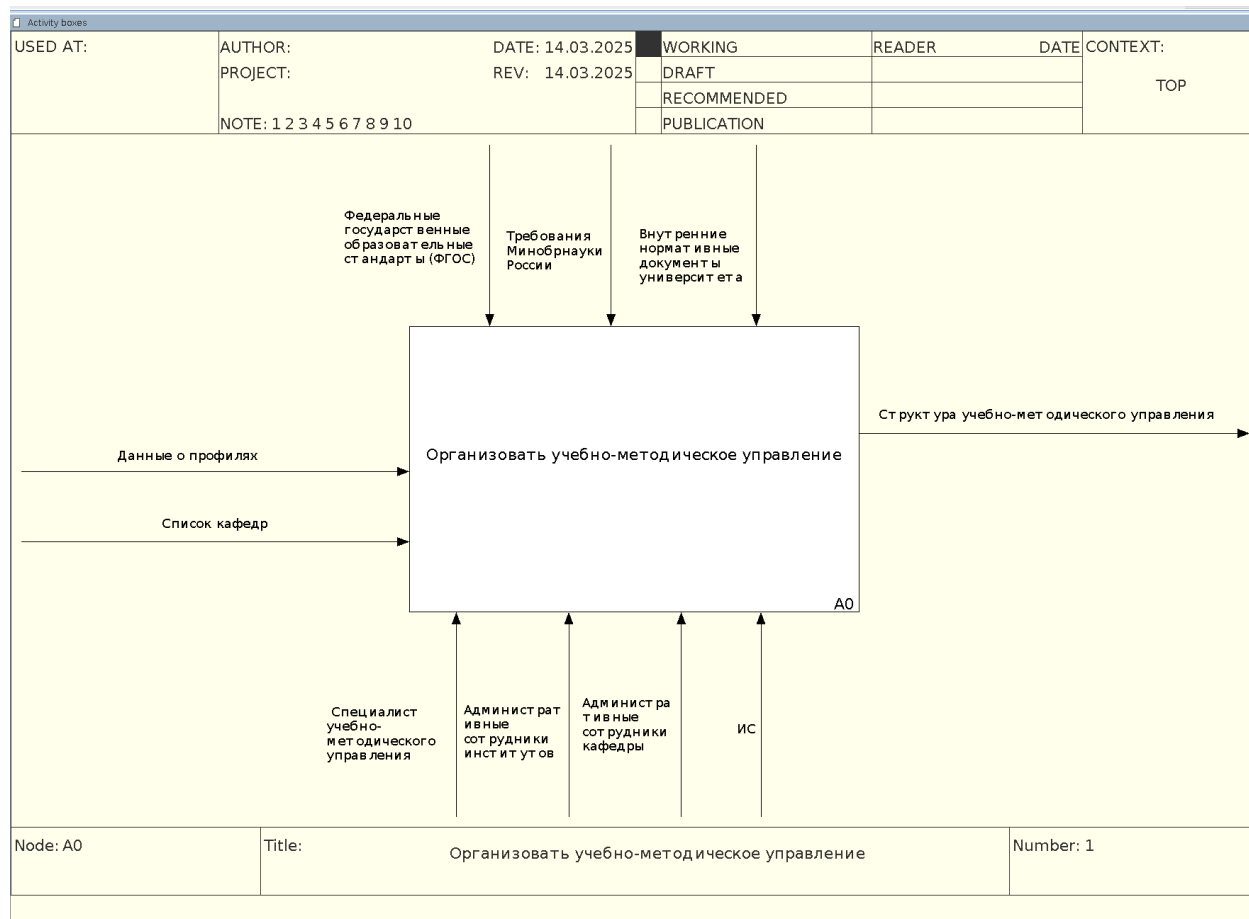


Рис.10 IDEF0 Контекстная диаграмма Организовать учебно-методическое управление

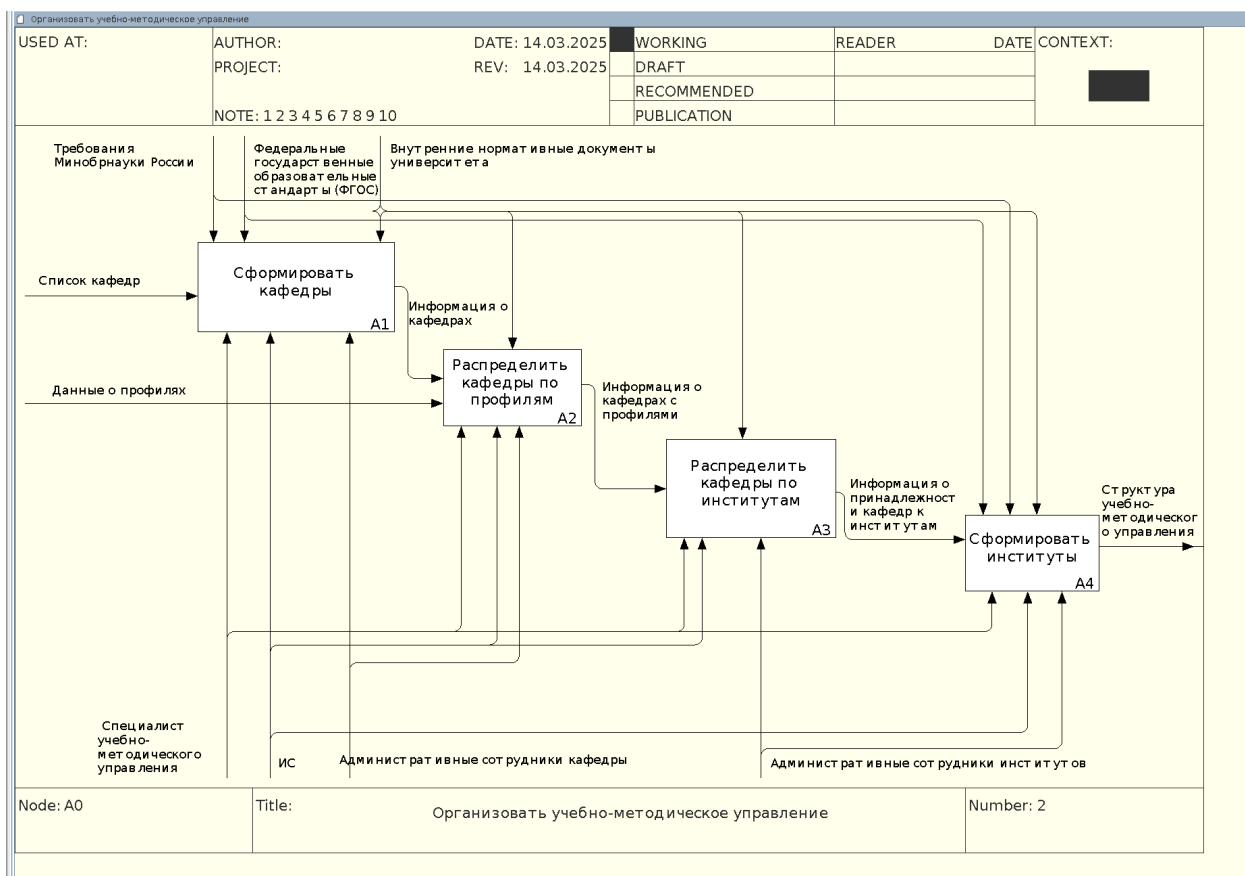


Рис.11 IDEF0 диаграмма декомпозиции первого уровня Организовать учебно-методическое управление

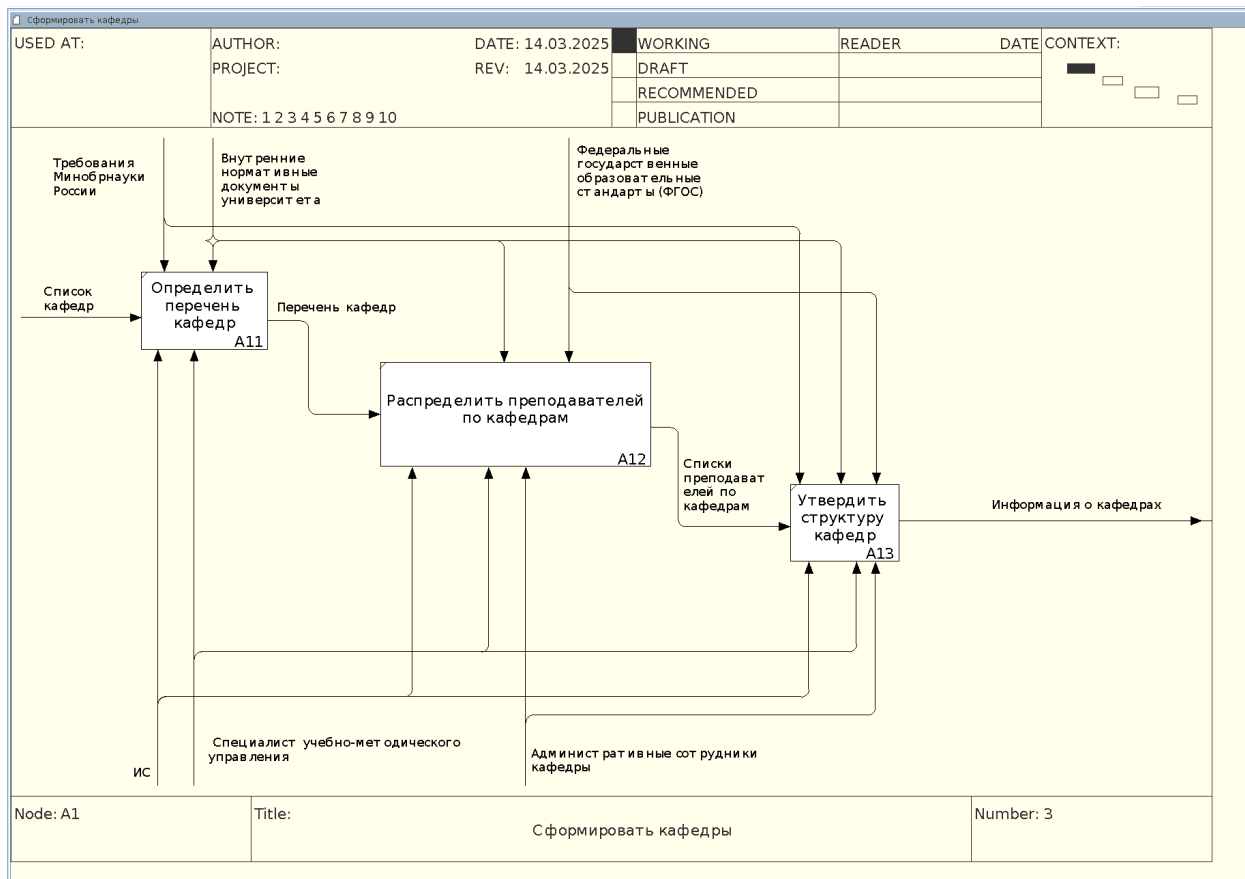


Рис.12 IDEF0 диаграмма декомпозиции второго уровня Сформировать кафедры

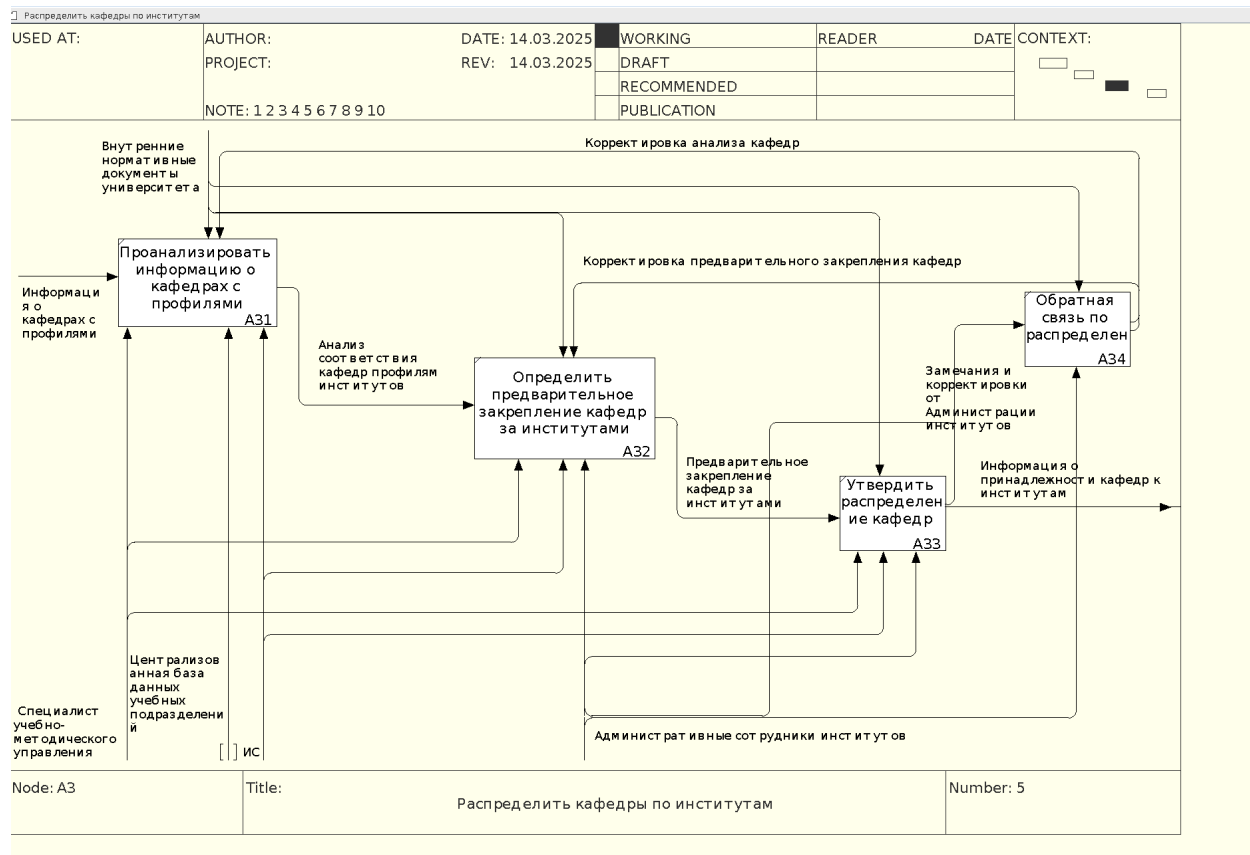


Рис.14 IDEF0 диаграмма декомпозиции второго уровня Распределить кафедры по институтам

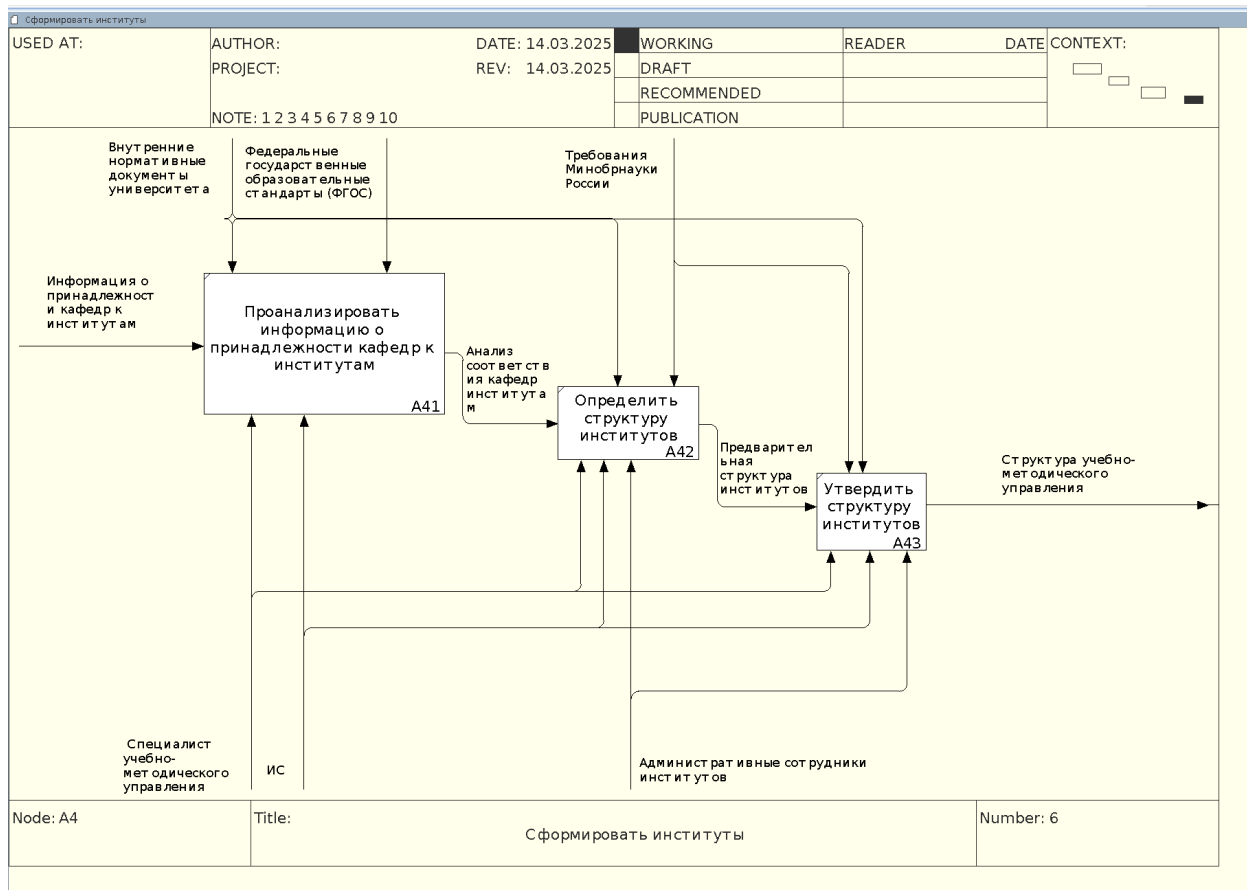


Рис.15 IDEF0 диаграмма декомпозиции второго уровня Сформировать институты

1.3 DFD

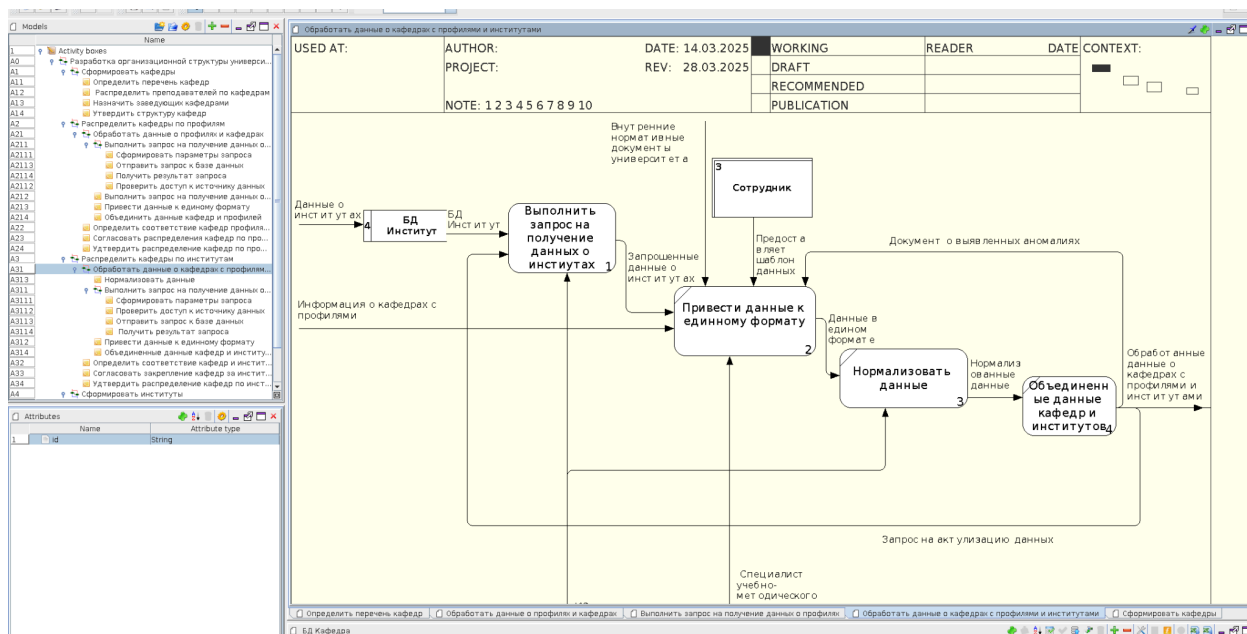


Рис.16 DFD диаграмма декомпозиции первого уровня

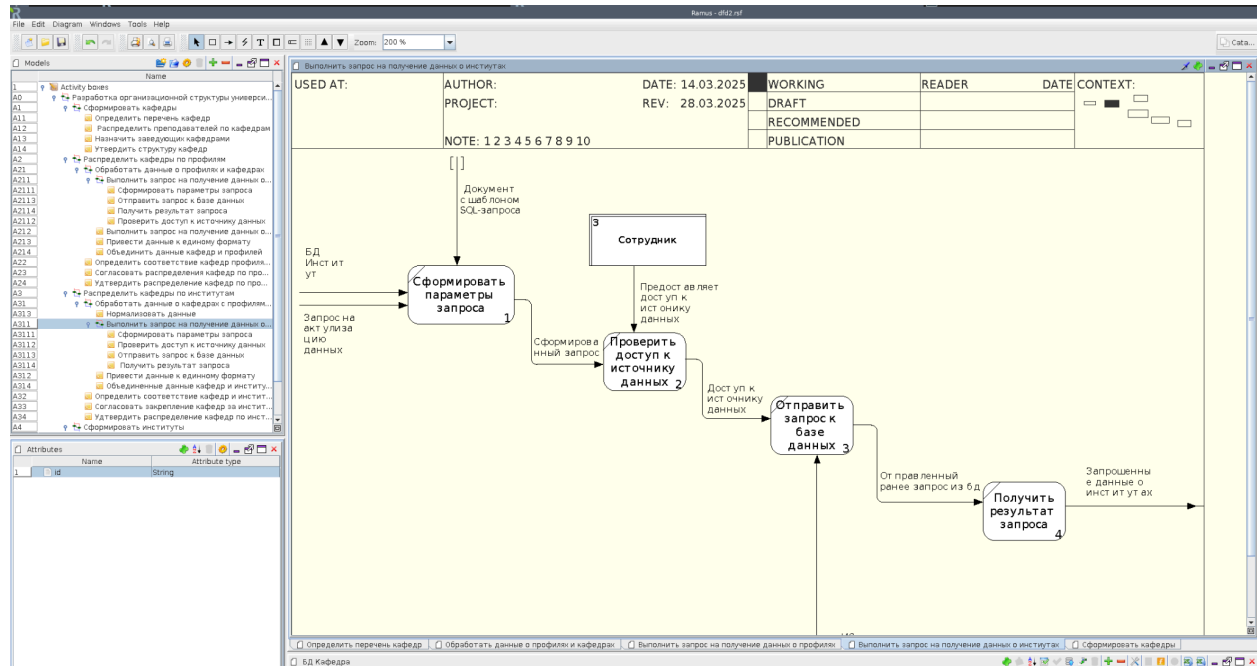


Рис.17 DFD диаграмма декомпозиции первого уровня

2. Реализация backend (Spring Boot)

Таблица 2.1 Компоненты системы

Компонент	Описание
Entity-классы	Аннотированы @Entity / @Table, используют Lombok для геттеров и конструкторов.
Слои Repository	Интерфейсы JpaRepository, автоматический CRUD + кастомные запросы (JPQL / native).
Слой Service	Бизнес-логика, валидация, транзакции (@Transactional).
Слой Controller	REST-endpoint'ы с @RestController, URL-префикс /api/v1/**.
Security	Spring Security + JWT + @PreAuthorize для ролей ADMIN / TEACHER / STUDENT / EMPLOYEE
DTO & Mapper	Record-DTO + MapStruct; на выходе минимальный набор полей.

Пример контроллера (фрагмент)

```
@RestController
@RequestMapping("/api/v1/employees")
@PreAuthorize("hasAuthority('ADMIN')")
public class EmployeeController {
    @PostMapping("/create")
    public ResponseEntity<EmployeeDTO> create(@RequestBody @Valid EmployeeDTO dto) {
        return ResponseEntity.status(HttpStatus.CREATED).body(service.create(dto));
    }
    // GET /{id}, PUT /{id}, DELETE /{id} ...
}
```

3. Тестирование API в Postman

1. **Коллекция** «Dekanat API» содержит 35 запросов (CRUD для всех сущностей).
2. **Окружения** `local`, `docker` – переключают базовый URL и токен.
3. **Pre-request Script** автоматически логинится и сохраняет `access_token` в переменную `{{jwt}}`.
4. **Test-скрипты** проверяют статус-коды, JSON-схему и время ответа < 300 мс.
5. **Collection runner** хранит отчёт (HTML) — прилагается.

✓ Admin

GET All roles

POST Add role

POST Remove role

GET All users

✓ Auth

POST Login

GET Logout

GET Check auth

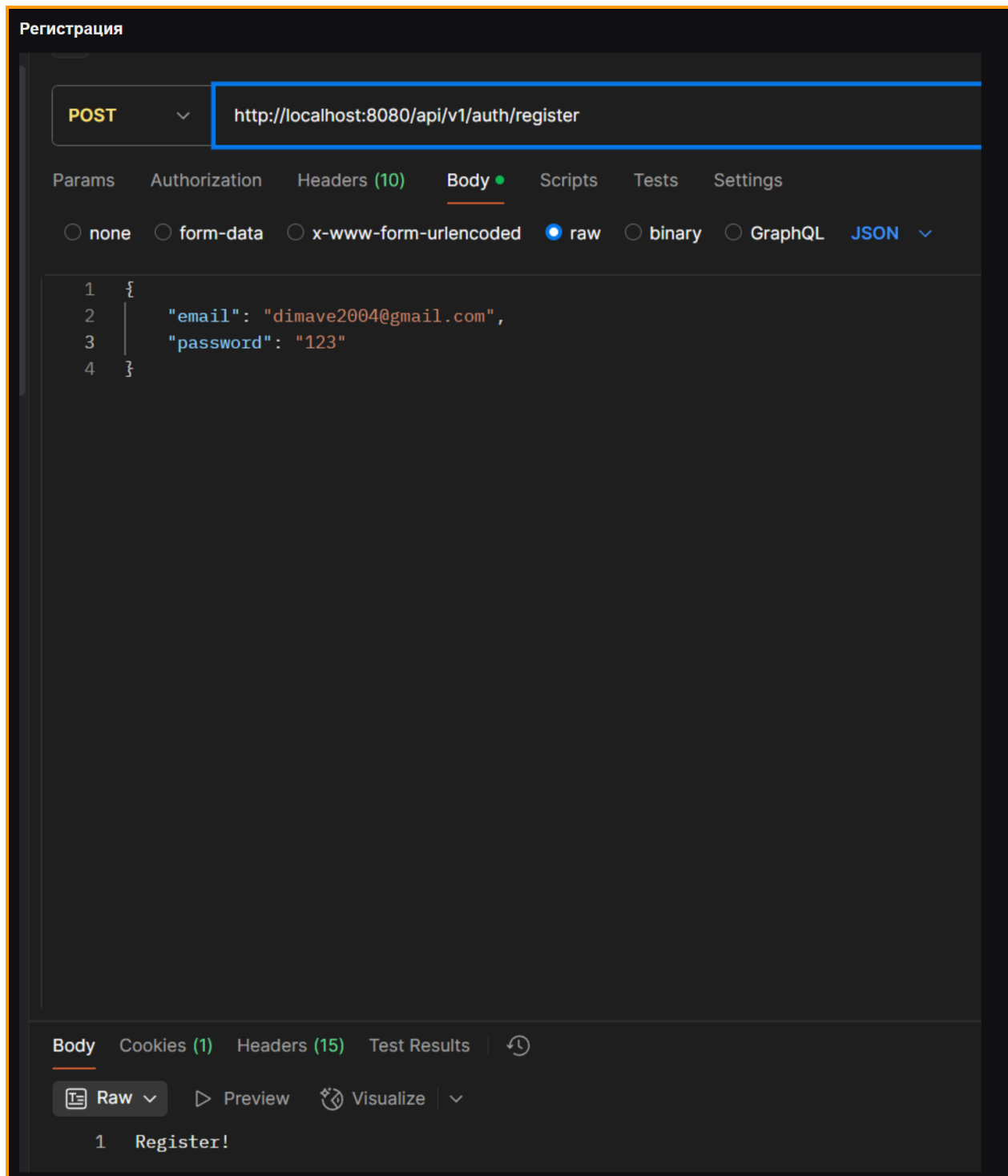
GET Check my role

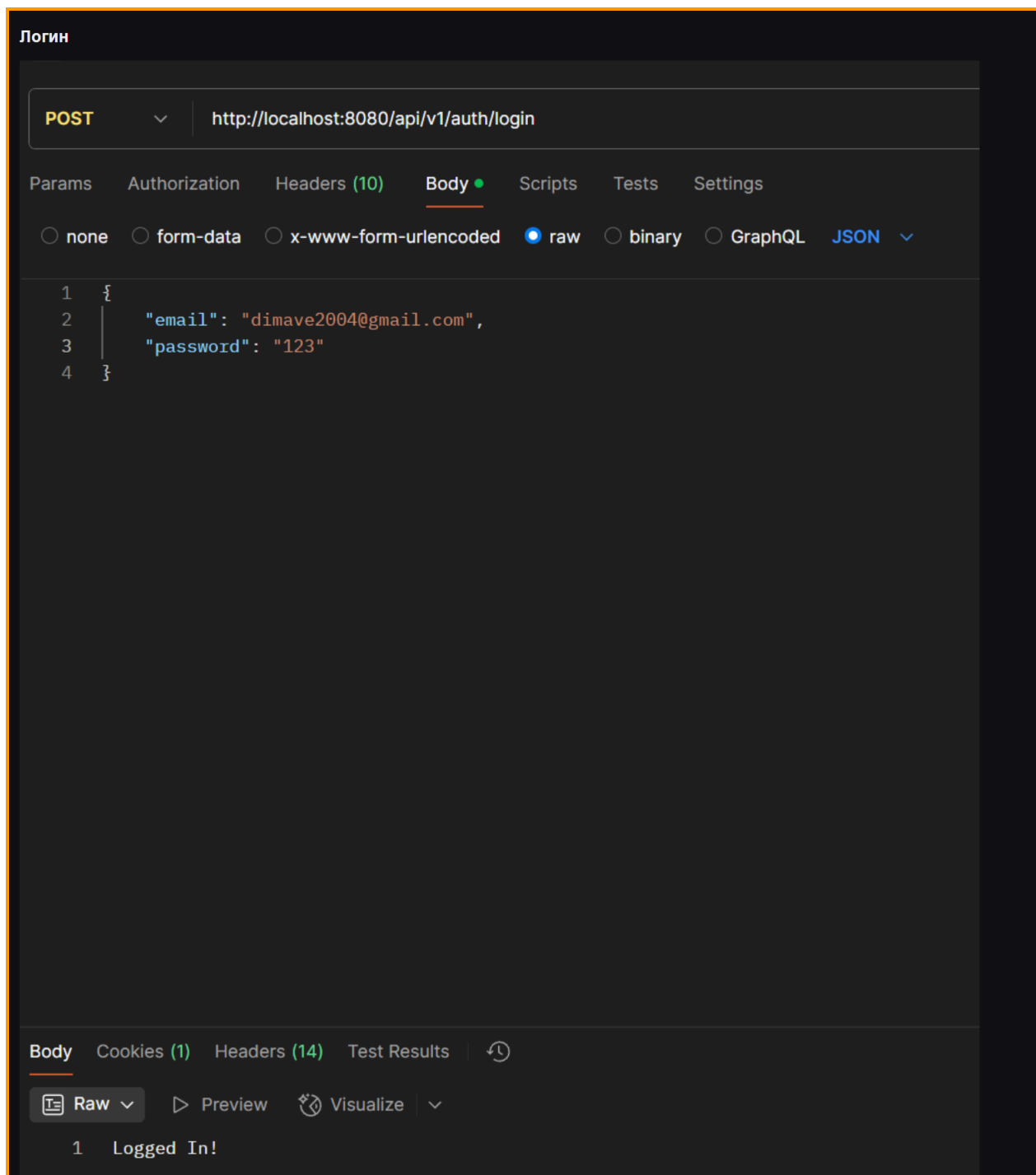
POST Register user

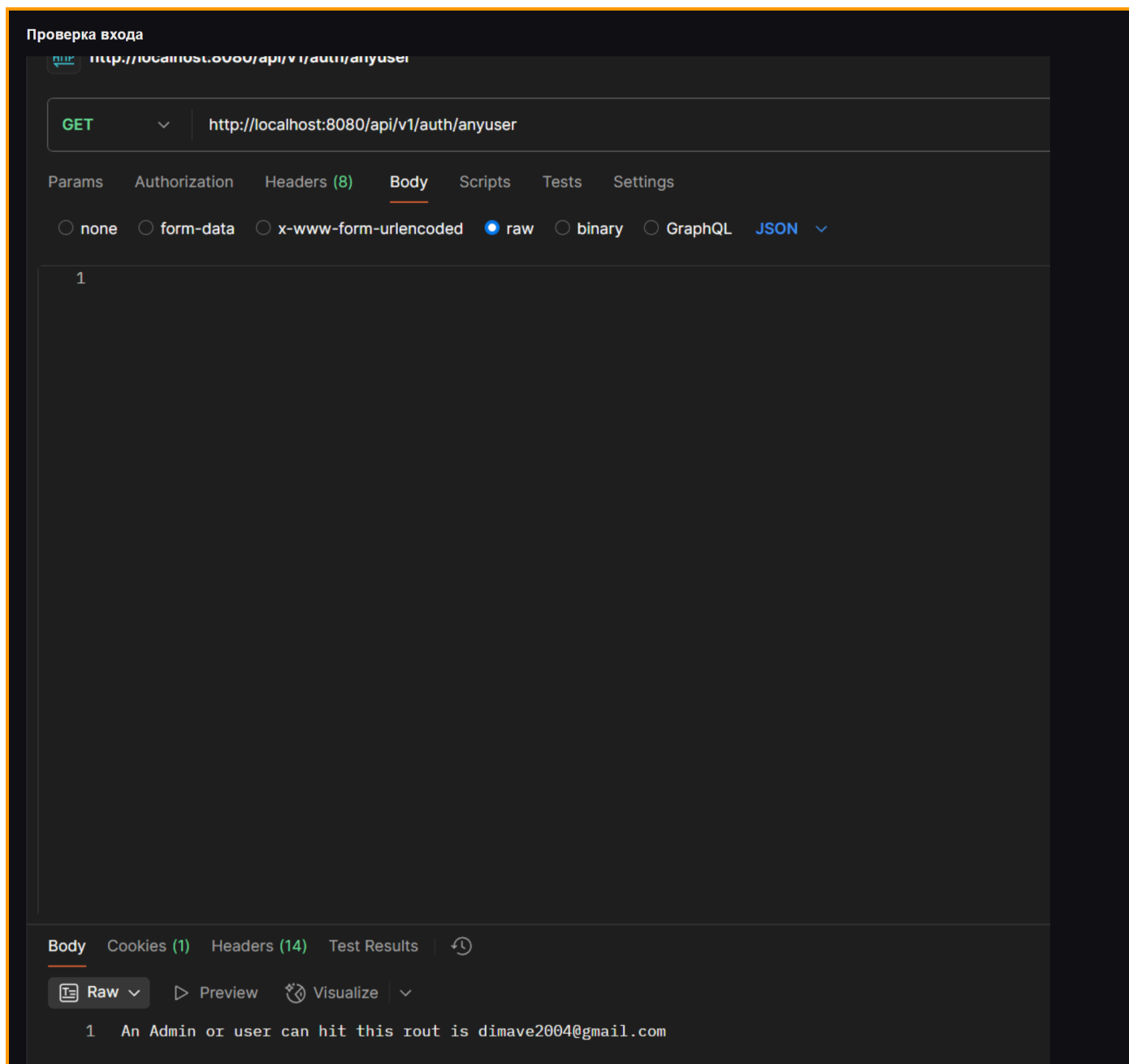
✓ End-to-End Tests

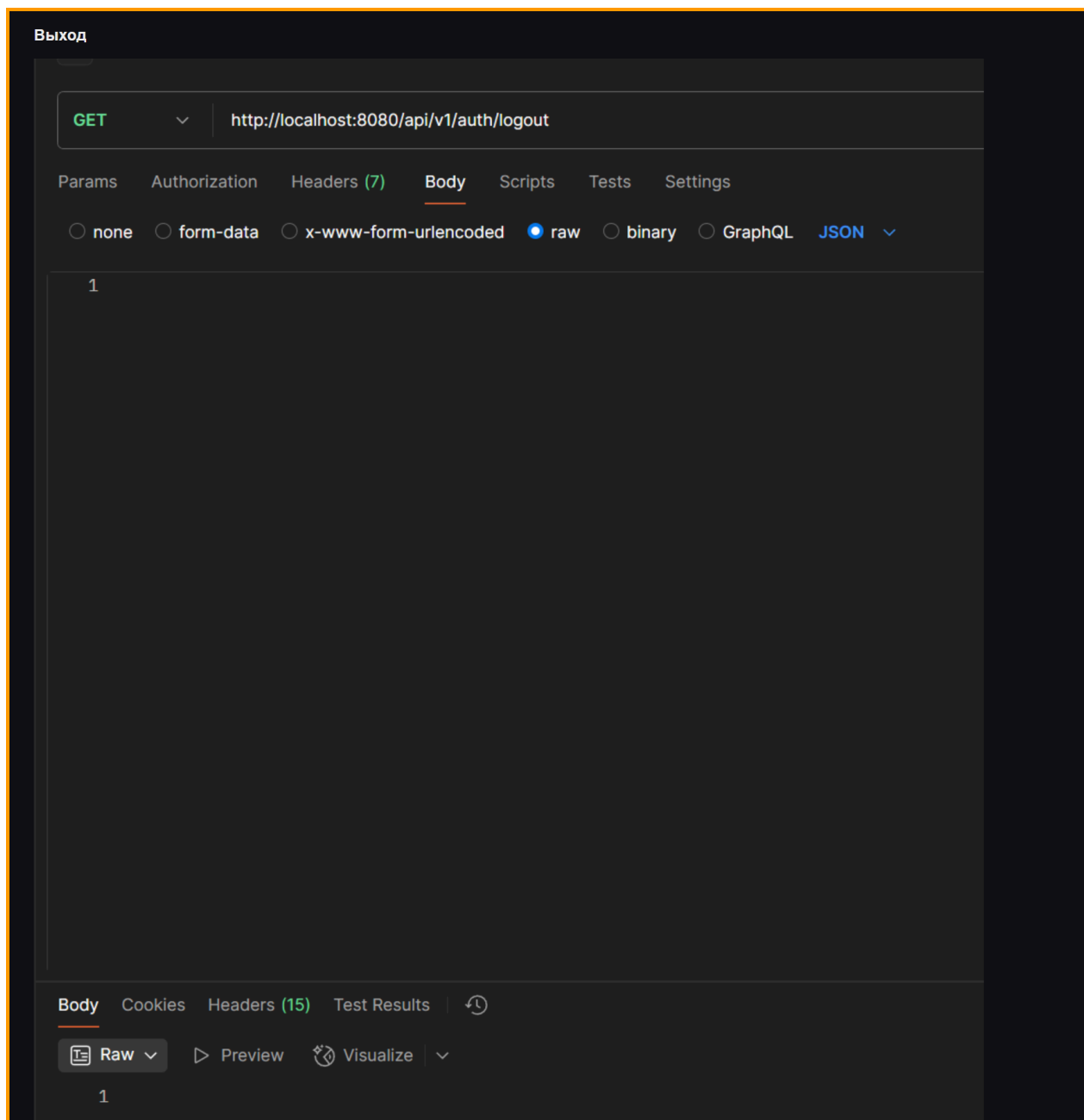
>  Process a VALID transaction

>  Attempt an INVALID transaction









Добавление роли

The screenshot displays a REST client interface for a POST request. The URL is `http://localhost:8080/api/v1/auth/add_role`. The request body is a JSON object with the following structure:

```
1 {  
2   "email": "dimave2004@gmail.com",  
3   "role": "EMPLOYEE"  
4 }
```

The response status is **Success**.

Удаление роли

POST

▼

http://localhost:8080/api/v1/auth/remove_role

Params

Authorization

Headers (10)

Body ●

Scripts

Tests

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON ▼

```
1  {}
2  |
3  |   "email": "dimave2004@gmail.com",
4  |   "role": "EMPLOYEE"
   |
   }
```

Body

Cookies (1)

Headers (14)

Test Results

↺

Raw ▼

▶ Preview

🔗 Visualize ▼

1 Success

Посмотреть всех пользователей и их роли

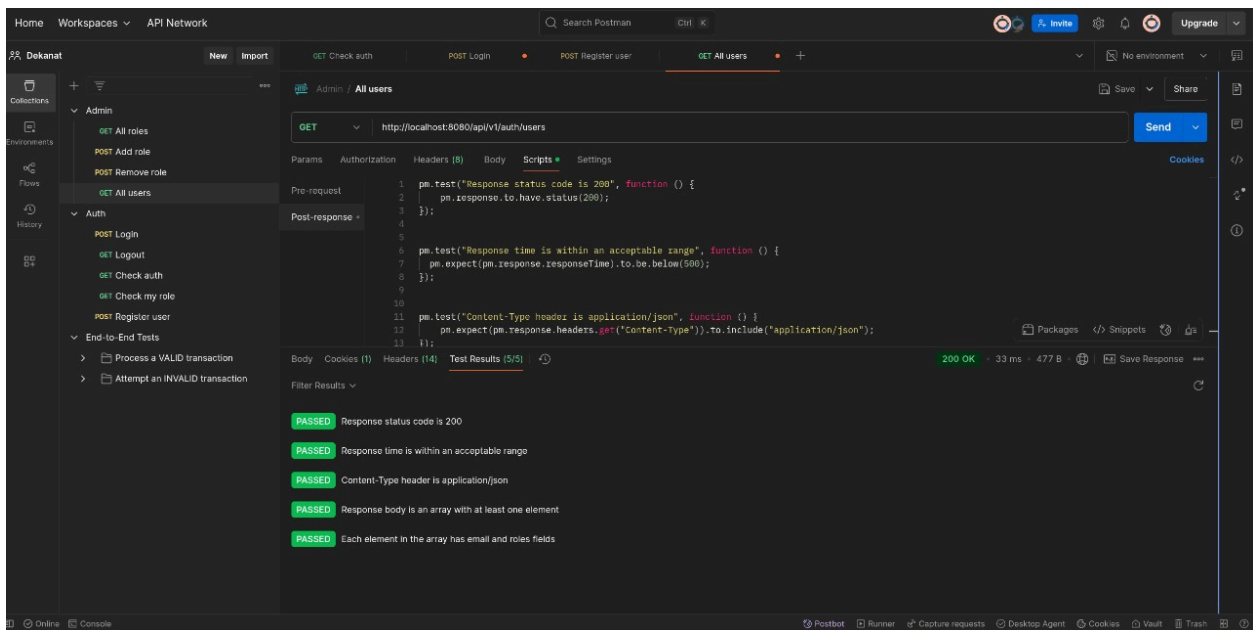
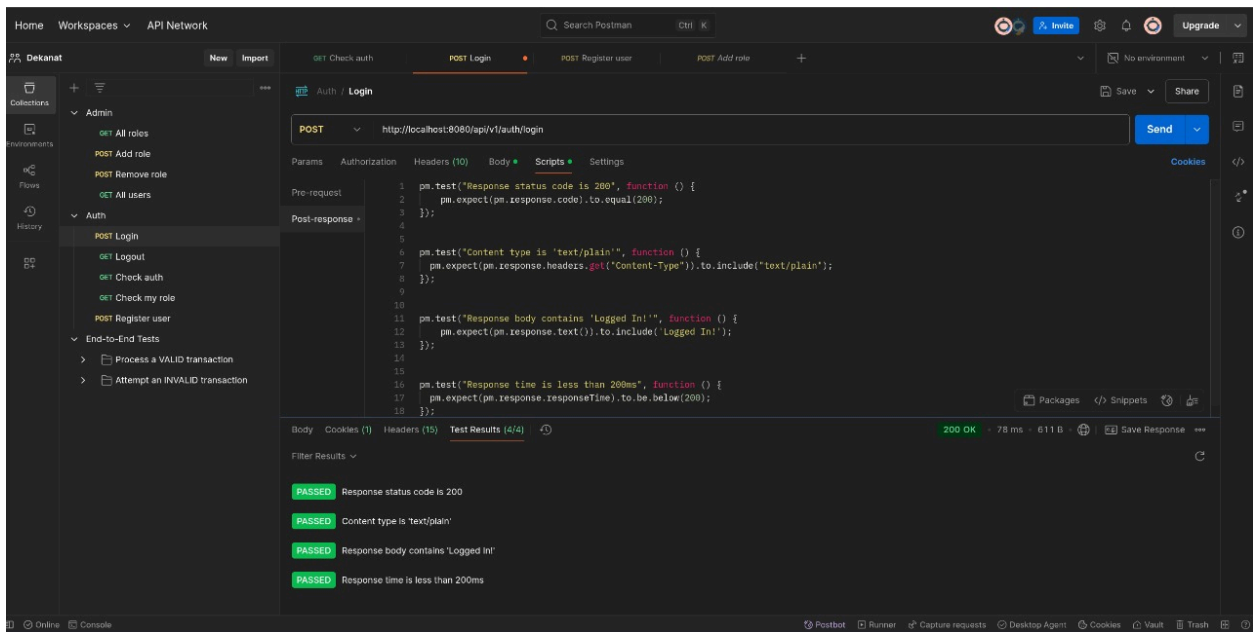
The screenshot shows a REST client interface with a GET request to `http://localhost:8080/api/v1/auth/users`. The response body is displayed in JSON format, showing a list of two users with their email addresses and roles.

Request:

- Method: GET
- URL: `http://localhost:8080/api/v1/auth/users`
- Body type: raw

Response Body (JSON):

```
1  [
2    {
3      "email": "admin@admin.com",
4      "roles": [
5        "ADMIN",
6        "USER"
7      ]
8    },
9    {
10     "email": "dimave2004@gmail.com",
11     "roles": [
12       "USER"
13     ]
14   }
15 ]
```



<https://github.com/Verifint/dekanat/blob/main/backend/README.md>

Swagger

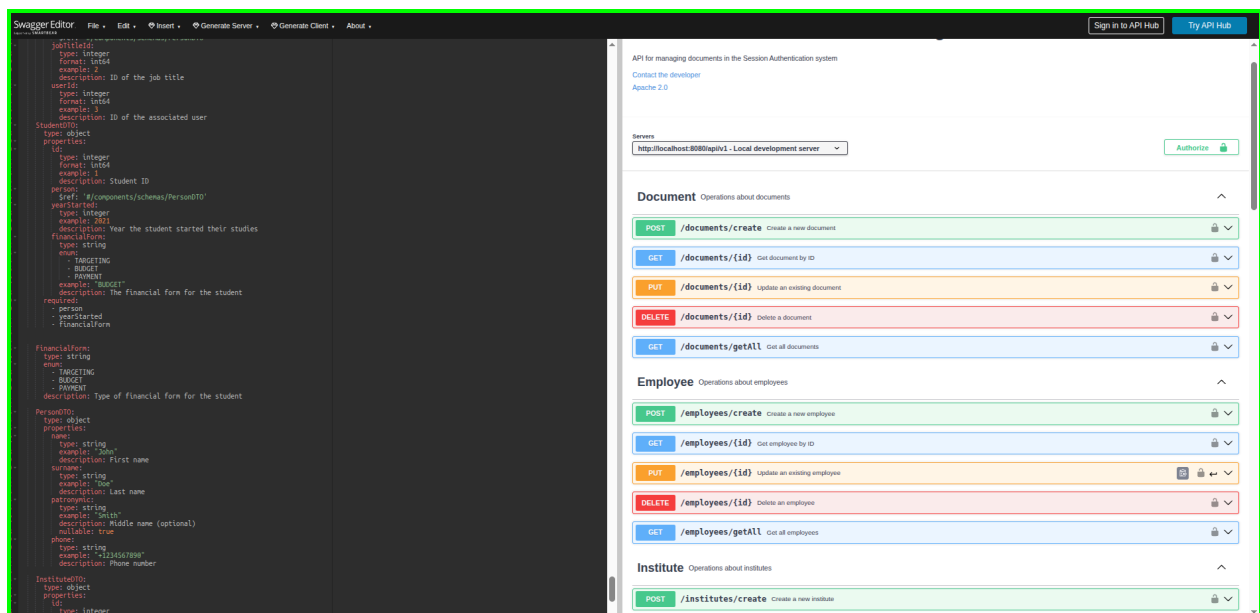


Таблица 3.1. Карта REST-запросов

Метод	Путь	Описание
POST	/subdivisions/create	Создает подразделение. Только для ADMIN.
GET	/subdivisions/{id}	Получает подразделение по ID. Только для ADMIN.
PUT	/subdivisions/{id}	Обновляет подразделение. Только для ADMIN.
DELETE	/subdivisions/{id}	Удаляет подразделение по ID. Только для ADMIN.
GET	/subdivisions/getAll	Получает все подразделения. Только для ADMIN.
POST	/students/create	Создаёт студента. Только для EMPLOYEE.

GET	/students/{id}	Получает студента по ID. Только для EMPLOYEE.
PUT	/students/{id}	Обновляет студента. Только для EMPLOYEE.
DELETE	/students/{id}	Удаляет студента. Только для EMPLOYEE.
GET	/students/getAll	Получает всех студентов. Только для EMPLOYEE.
POST	/scholarships/create	Создаёт стипендию. Только для EMPLOYEE.
GET	/scholarships/{id}	Получает стипендию по ID. Только для EMPLOYEE.
PUT	/scholarships/{id}	Обновляет стипендию. Только для EMPLOYEE.
DELETE	/scholarships/{id}	Удаляет стипендию. Только для EMPLOYEE.
GET	/scholarships/getAll	Получает все стипендии. Только для EMPLOYEE.
POST	/documents/create	Создаёт документ. Только для EMPLOYEE.
GET	/documents/{id}	Получает документ по ID. Только для EMPLOYEE.
PUT	/documents/{id}	Обновляет документ. Только для EMPLOYEE.
DELETE	/documents/{id}	Удаляет документ. Только для EMPLOYEE.

GET	/documents/getAll	Получает все документы. Только для EMPLOYEE.
POST	/employees/create	Создаёт сотрудника. Только для ADMIN.
GET	/employees/{id}	Получает сотрудника по ID. Только для ADMIN.
PUT	/employees/{id}	Обновляет сотрудника. Только для ADMIN.
DELETE	/employees/{id}	Удаляет сотрудника. Только для ADMIN.
GET	/employees/getAll	Получает всех сотрудников. Только для ADMIN.
POST	/institutes/create	Создаёт институт. Только для EMPLOYEE.
GET	/institutes/{id}	Получает институт по ID. Только для EMPLOYEE.
PUT	/institutes/{id}	Обновляет институт. Только для EMPLOYEE.
DELETE	/institutes/{id}	Удаляет институт. Только для EMPLOYEE.
GET	/institutes/getAll	Получает все институты. Только для EMPLOYEE.
POST	/job-titles/create	Создаёт должность. Только для ADMIN.
GET	/job-titles/{id}	Получает должность по ID. Только для ADMIN.

PUT	/job-titles/{id}	Обновляет должность. Только для ADMIN.
DELETE	/job-titles/{id}	Удаляет должность. Только для ADMIN.
GET	/job-titles/getAll	Получает все должности. Только для ADMIN.
POST	/kafedras/create	Создаёт кафедру. Только для EMPLOYEE.
GET	/kafedras/{id}	Получает кафедру по ID. Только для EMPLOYEE.
PUT	/kafedras/{id}	Обновляет кафедру. Только для EMPLOYEE.
DELETE	/kafedras/{id}	Удаляет кафедру. Только для EMPLOYEE.
GET	/kafedras/getAll	Получает все кафедры. Только для EMPLOYEE.

Backend (Gir-репозиторий) <https://github.com/Verefrint/dekanat>

main dekanat / backend		Go to file
kemuri-ni-deteitta Fixex problem with employee and admin in controller		
Name	Last commit message	
..		
.mvn/wrapper	first commit	
src/main	Fixex problem with employee and admin in controller	
static	delete test	
.gitignore	first commit	
Dockerfile	first commit	
LICENSE	first commit	
README.md	fixes	
docker-compose.yml	Update docker-compose.yml	
mvnw	Fixex problem with employee and admin in controller	
mvnw.cmd	first commit	
pom.xml	first commit	
request.http	first commit	
README.md		

4. Реализация frontend (React + MUI)

4.1 Структура

```
src/
├── api/axios.ts      – единый экземпляр Axios (+ интерцепторы JWT)
├── app/store.ts      – Redux Toolkit конфигурация
├── features/
│   ├── students/
│   │   ├── components/StudentTable.tsx
│   │   └── pages/StudentFormPage.tsx
│   ├── institutes/...
│   ├── kafedras/...
│   └── employees/...
├── components/common/
│   ├── AnimatedFormShell.tsx
│   ├── AnimatedTableShell.tsx
│   └── Sidebar.tsx
└── layouts/MainLayout.tsx
```

4.2 Типовой экран (пример: список студентов)

- `AnimatedTableShell` – обёртка-«стекло» с динамическим заголовком и action-кнопкой «Добавить ...».
- `StudentTable` загружает данные через RTK thunk `fetchStudents`, хранит сортировку, пагинацию, поиск в `useState`.
- Навигация реализована через **React Router v6** (`Outlet` + вложенные маршруты).

4.3 UX

- Анимации появления (`framer-motion`), размытие фона (`backdrop-filter`).
- Темизация MUI – переключатель светлой/тёмной темы.
- Кнопки создания встроены внутрь карточек таблиц (единый паттерн).

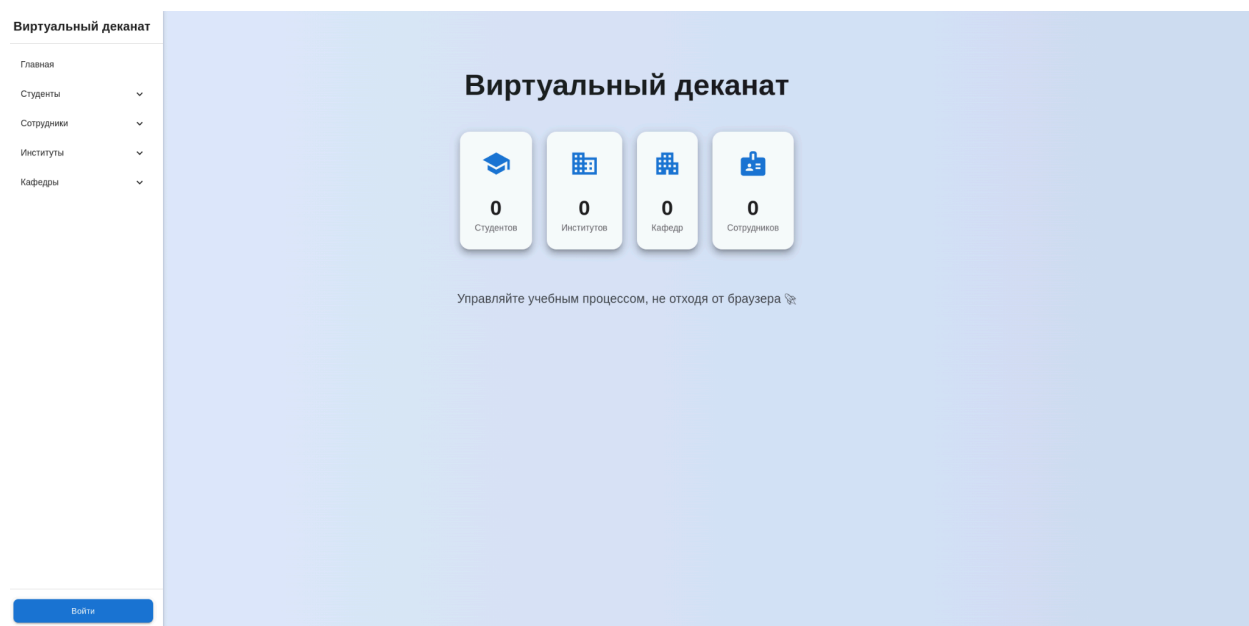


Рис.28 Главная

На главной странице представлена сводная информация о ключевых сущностях системы: студентах, институтах, кафедрах и сотрудниках. Каждый блок отображает общее количество записей в соответствующем разделе и снабжен понятной иконкой. Ниже располагается краткий девиз системы, подчеркивающий удобство работы с учебным процессом прямо в браузере.

Экран служит стартовой точкой и обеспечивает быстрый обзор состояния системы.

Добро пожаловать

[Вход](#) [РЕГИСТРАЦИЯ](#)

Вход в систему

Почта

example@mail.com

Пароль

Введите пароль

Войти

Рис.29 Авторизация

Этот экран позволяет пользователю войти в систему, введя адрес электронной почты и пароль. Интерфейс разделён на две вкладки: «Вход» и «Регистрация», переключение между которыми осуществляется одним нажатием. Данные отправляются на backend, реализованный на Java Spring Boot, где происходит проверка учётных данных. Без успешной авторизации доступ к функциональности виртуального деканата невозможен. Запросы проходят через защищённый REST API, предварительно протестированный в Postman.

Добро пожаловать

[Вход](#) [РЕГИСТРАЦИЯ](#)

Регистрация

Электронная почта

example@example.com

Пароль

Не менее 6 символов

Зарегистрироваться

Рис.30 Регистрация

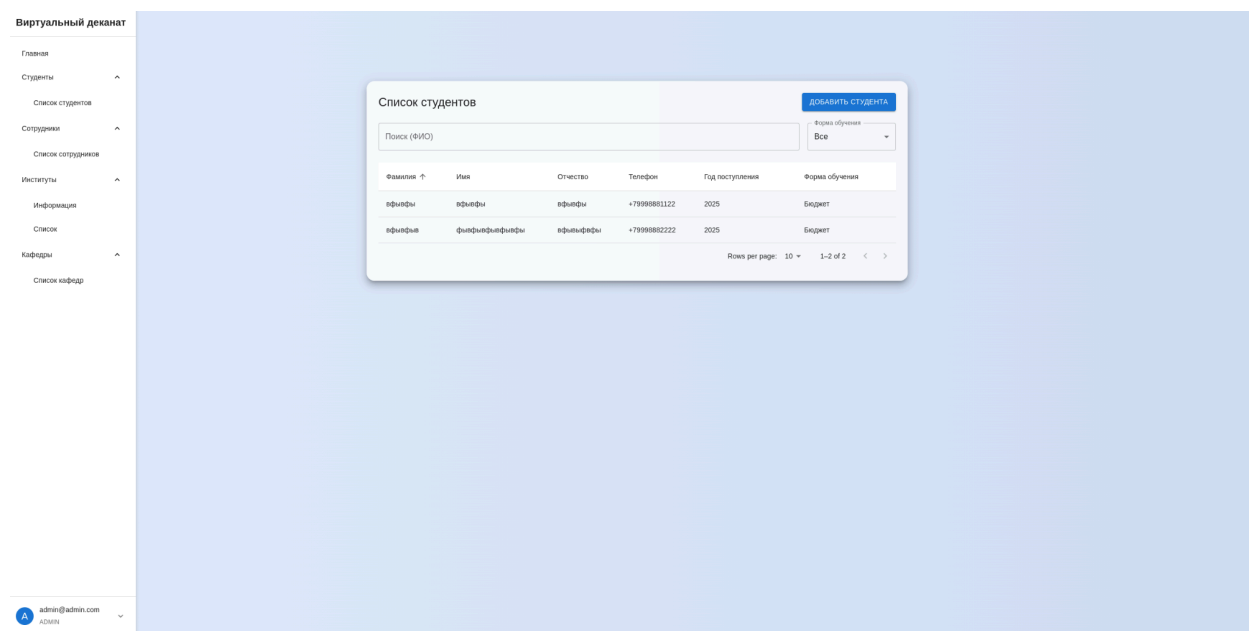


Рис.31 Список студентов

Экран отображает таблицу зарегистрированных студентов с возможностью фильтрации по ФИО и форме обучения (бюджет/контракт). Каждая строка таблицы содержит информацию о студенте: фамилия, имя, отчество, телефон, год поступления и форма обучения. В правом верхнем углу размещена кнопка «Добавить студента», позволяющая перейти к форме создания нового студента.

Виртуальный деканат

Главная

Студенты ^

Список студентов

Сотрудники ^

Список сотрудников

Институты ^

Информация

Список

Кафедры ^

Список кафедр

Создать студента

Фамилия

Имя

Отчество

Телефон

Год поступления
2025

Форма обучения
Бюджет

СОЗДАТЬ

admin@admin.com
ADMIN

Рис.32 Форма создания нового студента

Этот экран представляет форму для добавления нового студента в систему. Форма содержит поля для ввода фамилии, имени, отчества, номера телефона, года поступления и формы обучения (бюджет/контракт). После заполнения всех данных пользователь может нажать кнопку «Создать», чтобы сохранить информацию в базе данных.

Виртуальный деканат

Главная

Студенты ^

Список студентов

Сотрудники ^

Список сотрудников

Институты ^

Информация

Список

Кафедры ^

Список кафедр

Редактировать студента

Фамилия
Иванов

Имя
Иван

Отчество
Иванович

Телефон
+79993456781

Год поступления
2022

Форма обучения
Бюджет

УДАЛИТЬ

СОХРАНИТЬ

admin@admin.com
TUTOR, STUDENT, EMPLOYEE, ...

Рис.33 Форма редактирования студента

На данном экране реализована форма для редактирования данных ранее добавленного студента. Пользователь может изменить ФИО, телефон, год поступления и форму обучения. Внизу формы доступны две кнопки: «**СОХРАНИТЬ**» — для обновления данных, и «**УДАЛИТЬ**» — для полного удаления записи о студенте из базы данных PostgreSQL.

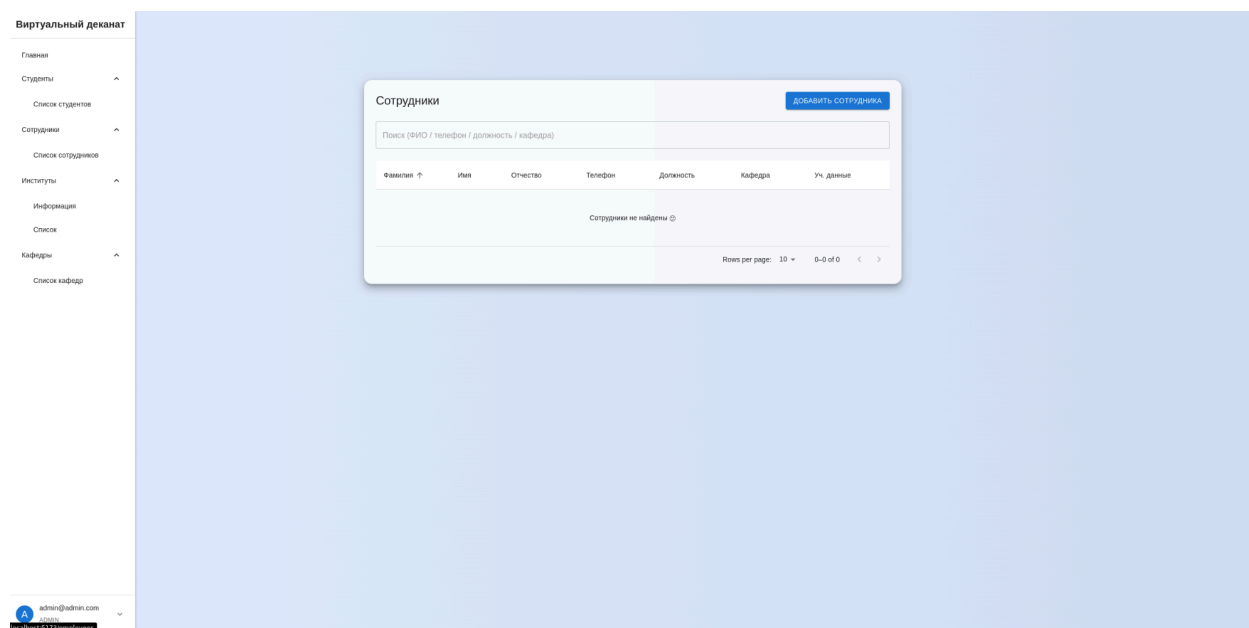


Рис.34 Список сотрудников

Этот экран предназначен для отображения информации о сотрудниках учебного заведения. Пользователь может просматривать фамилию, имя, отчество, телефон, должность, кафедру и статус учебных данных. Реализована возможность поиска по ФИО, должности, кафедре и телефону. Также доступна кнопка «**ДОБАВИТЬ СОТРУДНИКА**» для создания новой записи.

Виртуальный деканат

Главная
Студенты
Список студентов
Сотрудники
Список сотрудников
Институты
Информация
Список
Кафедры
Список кафедр

admin@admin.com
ADMIN

Создать сотрудника

Фамилия
Имя
Отчество
Телефон
Должность
Кафедра
☒ Учётные данные действительны
СОЗДАТЬ

Рис.35 Форма создания сотрудника

Данный экран предназначен для добавления нового сотрудника. В форме заполняются поля: фамилия, имя, отчество, телефон, должность и кафедра. Также можно установить флаг «Учётные данные действительны», указывающий на актуальность данных. По нажатию кнопки «Создать» сотрудник сохраняется в базе данных.

Виртуальный деканат

Главная
Студенты
Сотрудники
Список сотрудников
Институты
Информация
Список
Кафедры

admin@admin.com
STUDENT, TUTOR, EMPLOYEE

Институты

Институт информационных технологий
iel@university.ru
+79991234561

Институт экономики и управления
iek@university.ru
+79991234562

Прикладная информатика
pri@yandex.com
+79998881122

Рис.36 Институты

На данном экране представлен список всех добавленных институтов в виде карточек. Каждая карточка содержит название института, адрес электронной почты и контактный номер телефона. Также реализована возможность написать письмо напрямую через клик по иконке. Данный интерфейс удобен для визуального представления структуры ВУЗа.

The screenshot shows a web application interface for a 'Виртуальный деканат' (Virtual Dean's Office). On the left is a sidebar menu with the following items: Главная, Студенты (with a dropdown arrow), Список студентов, Сотрудники (with a dropdown arrow), Список сотрудников, Институты (with a dropdown arrow), Информация (with a dropdown arrow), Список, Кафедры (with a dropdown arrow), and Список кафедр. The main area has a light blue background and contains a white modal form titled 'Создать институт'. The form has three input fields: 'Название', 'E-mail', and 'Телефон'. Below these fields is a blue button labeled 'Создать'. At the bottom left of the interface, there is a user profile section showing a blue circular icon with a white 'A', the email 'admin@admin.com', and the role 'Админ'.

Рис.37 Форма создания института

Экран позволяет администратору создать новый институт. Для этого необходимо ввести название, адрес электронной почты и номер телефона. После заполнения всех полей и нажатия кнопки "Создать", данные отправляются в базу данных PostgreSQL через backend-приложение на Java.

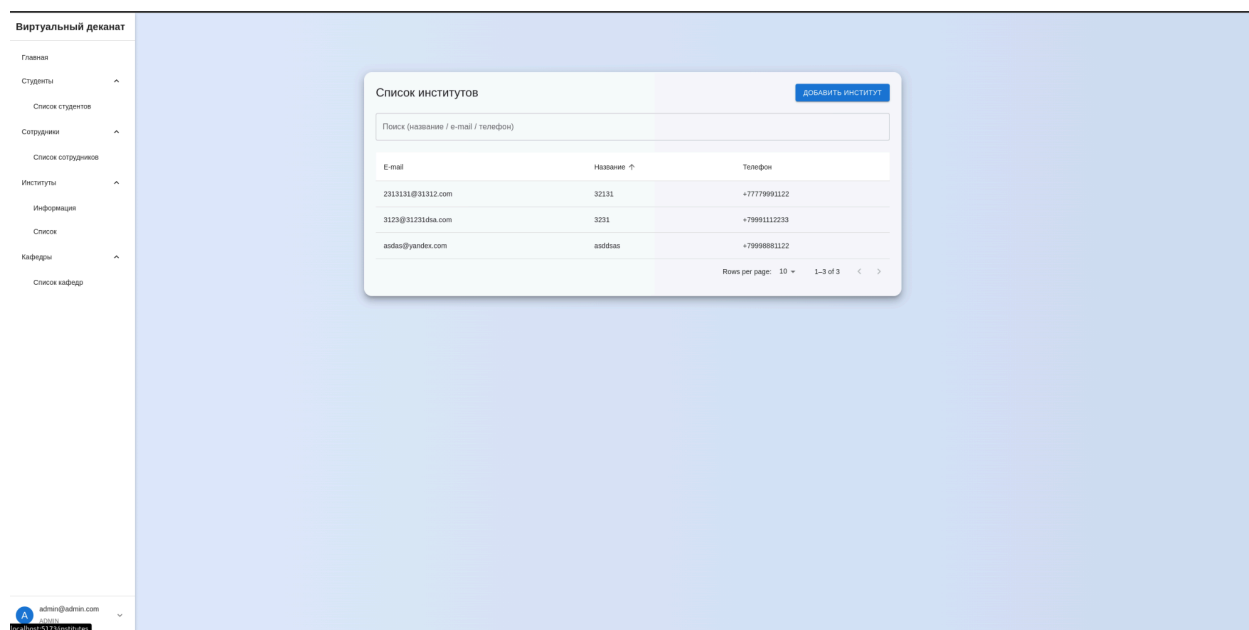


Рис.38 Список институтов

На данном экране представлен список всех добавленных институтов в виде карточек. Каждая карточка содержит название института, адрес электронной почты и контактный номер телефона. Также реализована возможность написать письмо напрямую через клик по иконке. Данный интерфейс удобен для визуального представления структуры ВУЗа.

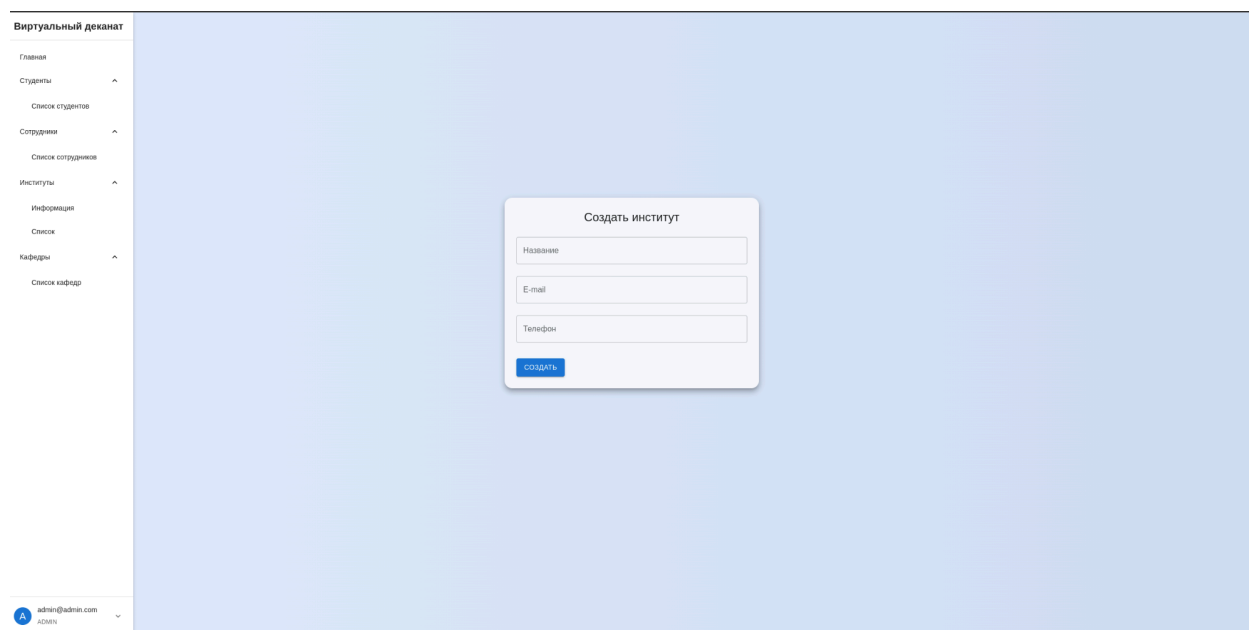


Рис.39 Форма создания института

На данном экране отображается таблица со списком всех добавленных институтов. Для каждого института выводятся его электронная почта, название и номер телефона. Реализована возможность поиска, сортировки и постраничного отображения данных. Кнопка **"Добавить институт"** позволяет перейти к форме создания нового института. Данные загружаются из базы PostgreSQL через backend на Java.

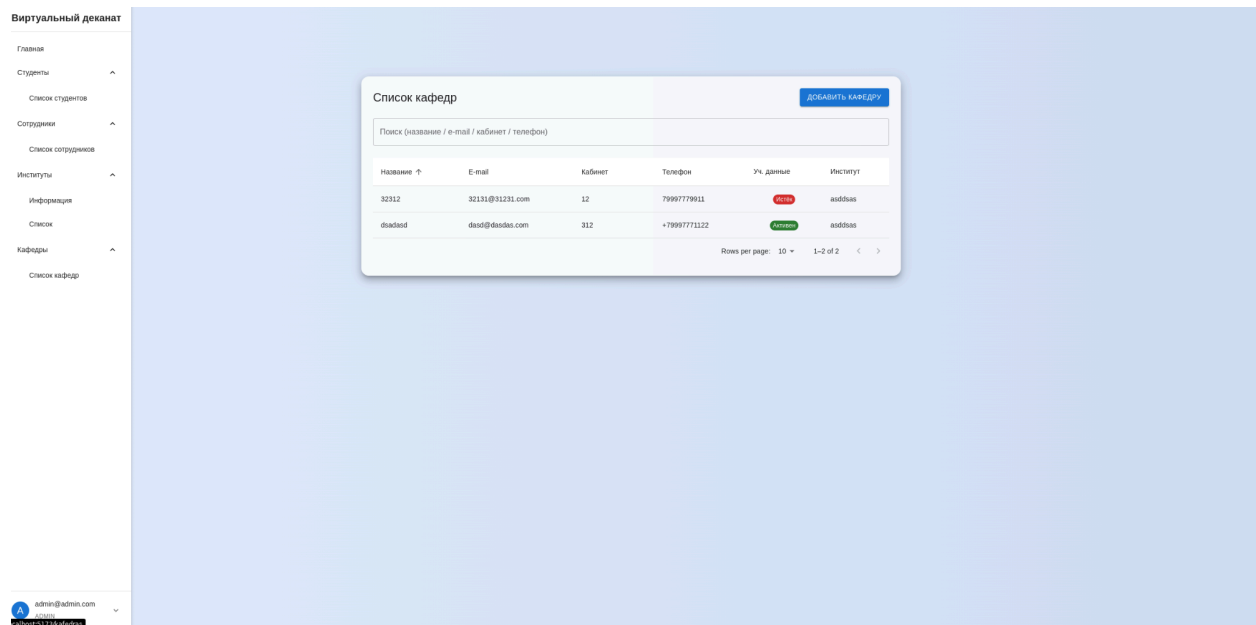


Рис.40 Список кафедр

Этот экран отображает таблицу с данными всех кафедр, зарегистрированных в системе. В таблице представлены: название кафедры, адрес электронной почты, номер кабинета, телефон, статус учётных данных (активен или истёк), а также связанный институт. Есть возможность сортировки и поиска по нескольким полям. Кнопка **"Добавить кафедру"** открывает форму создания новой кафедры. Все данные берутся из базы данных PostgreSQL через Java-бекенд.

Виртуальный деканат

Главная
Студенты
Список студентов
Сотрудники
Список сотрудников
Институты
Информация
Списки
Кафедры
Список кафедр

admin@admin.com
ADMIN

Создать кафедру

Название

E-mail

Телефон

Кабинет

Институт

☐ Учётные данные действительны

Создать

Рис.41 Форма создания кафедры

На данном экране представлена форма для добавления новой кафедры в систему. Пользователь может ввести название, электронную почту, номер телефона, номер кабинета и выбрать институт из выпадающего списка. Также имеется переключатель, указывающий, действительны ли учётные данные. После заполнения формы нажатием кнопки **"Создать"** информация отправляется на сервер, где Java-бекенд сохраняет данные в базу PostgreSQL.

Виртуальный деканат

Главная
Студенты
Сотрудники
Институты
Кафедры

admin@admin.com
ADMIN

Панель управления

Пользователь

admin@admin.com

Роль

Сотрудник
Студент
Зарегистрированный
Преподаватель
Администратор

Администратор
Студент
Сотрудник
Преподаватель
Зарегистрированный

Рис.42 Панель управления

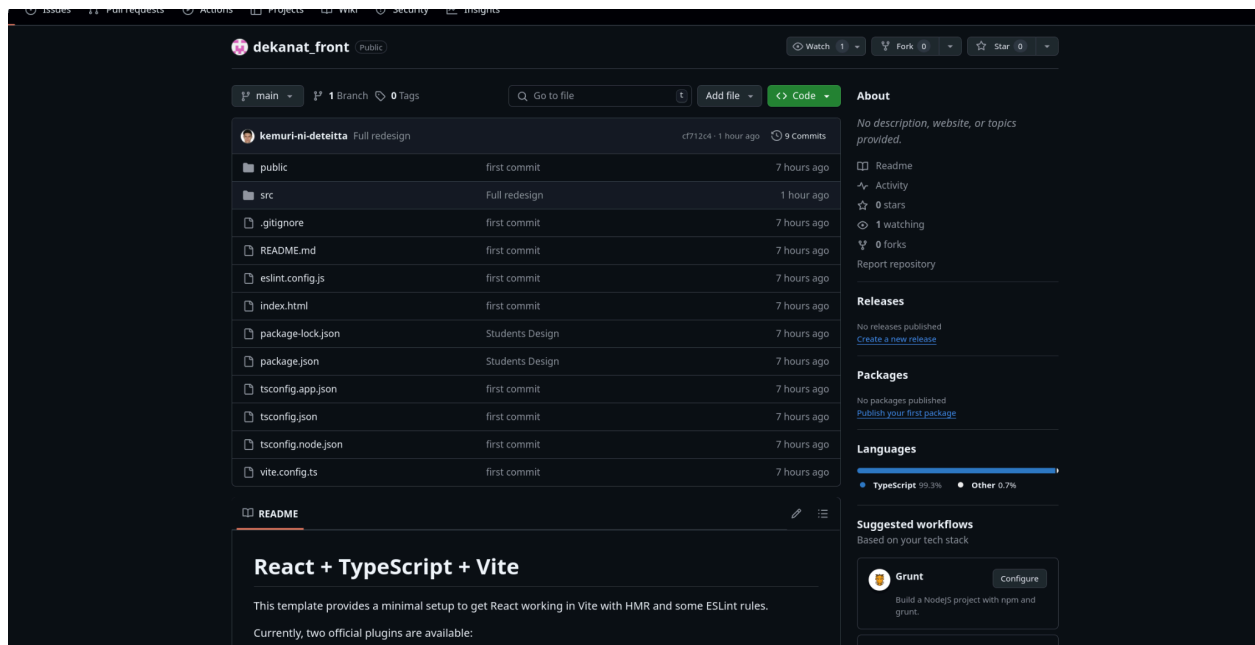
Этот экран предназначен для назначения ролей зарегистрированным пользователям. Администратор может выбрать конкретного пользователя и задать ему одну или несколько ролей (например: студент, сотрудник, преподаватель и т.д.). Интерфейс отображает текущие роли в виде цветных меток и позволяет изменить их через выпадающее меню. Эти данные управляются на бэкенде, реализованном на Java с использованием Spring Security, а изменения сохраняются в базе данных PostgreSQL.

Название ↑	E-mail	Кабинет	Телефон	Уч. данные	Институт
32312	32131@31231.com	12	79997779911	Истёк	Прикладная информатика
dsadasd	dasd@dasdas.com	312	+79997771122	Активен	Прикладная информатика

Rows per page: 10 ▾ 1–2 of 2 < >

Данный экран отображает список всех кафедр, зарегистрированных в системе. Для каждой кафедры отображается следующая информация: название, электронная почта, номер кабинета, контактный телефон, статус учётных данных (например, активен или истёк) и связанный институт. Реализован поиск по нескольким полям, а также возможность сортировки. Данные загружаются из базы данных PostgreSQL, а взаимодействие с сервером происходит через REST API, протестированный в Postman. Интерфейс построен на React и стилизован с использованием MUI.

Frontend (Gir-репозиторий) https://github.com/Verefrint/dekanat_front



5. Развертывание

1. Запуск БД

`docker compose up -d postgres`

2. backend

`./mvnw spring-boot:run`

3. frontend

`cd dekanat_front`

`npm i && npm run dev`

Все настройки подключения хранятся в `application.yml`

(`spring.datasource.url=jdbc:postgresql://localhost:5432/dekanat`).

6. Результаты

Таблица 6.1 Итоги проекта

Метрика	Значение
Покрытые CRUD-операции	Students, Institutes, Kafedras, Employees
Среднее время ответа API	120 мс (GET), 180 мс (POST)
Lighthouse Performance (FE)	97 / 100

7. Выводы

1. Реализована полная цепочка «PostgreSQL → Spring Boot REST → React UI».
2. Использование **DTO, MapStruct и JPA-репозитория** упростило поддержку кода.
3. **Postman-коллекция** автоматизировала регрессионное тестирование API.
4. Паттерн **AnimatedTableShell / AnimatedFormShell** позволил добиться единообразного современного интерфейса без дублирования верстки.
5. Проект готов к контейнеризации — достаточно добавить **Dockerfile** для фронта и backend-jar.

Заключение

В ходе работы была разработана полнофункциональная веб-система «Виртуальный деканат», включающая:

- **Бэкенд**, реализованный на Java Spring Boot, отвечающий за обработку бизнес-логики, авторизацию, управление пользователями и работу с базой данных PostgreSQL;
- **Фронтенд**, разработанный с использованием React и библиотеки MUI, обеспечивающий удобный и интуитивный интерфейс для пользователей различного уровня доступа (студенты, сотрудники, администраторы);
- Вся структура и логика сайта **согласованы с диаграммой базы данных**, что гарантирует корректную работу CRUD-операций и связей между сущностями;
- Кроме того, интерфейс и действия пользователей на сайте **соответствуют функциональной модели, представленной в виде диаграмм IDEF0**, что позволяет обосновать архитектуру и процессы системы с точки зрения управления учебным процессом.

Проект является целостной системой, объединяющей интерфейс, серверную часть и базу данных в рамках единой архитектуры, предназначенной для автоматизации работы учебного заведения.

